

Fondamenti di Cybersecurity

Appunti

Giovanni "Qua' Qua' dancer" Palma
Alex "Morbidelli^e WhatsApp" Basta

Contents

Chapter 1	Discrete Probability (a short reminder)	Page
1.1	Distribuzione di probabilità Distribuzioni particolari —	
1.2	Eventi Unione di eventi —	
1.3	Variabili aleatorie La variabile aleatoria uniforme — • Algoritmi randomizzati —	
1.4	Indipendenza	
1.5	XOR e la sua proprietà fondamentale	
1.6	Il paradosso del compleanno (birthday paradox)	
Chapter 2	Modular Arithmetic	Page
2.1	L'insieme $\mathbb{Z}_m$	
2.2	Congruenza	
2.3	Invertibilità e gcd	
Chapter 3	Introduzione e Cifrari Simmetrici	Page
3.1	Introduction to Cryptography Encryption Terminology — • Goals and Protocols — • Kerchoff's Principle and the Threat Model — • Symmetric Encryption — • Asymmetric Encryption — • Secure Communication — • But crypto can do much more —	
3.2	Cifrari classici Cifrario di Cesare / Shift cipher — • Monoalphabetic substitution cipher — • Affine cipher — • Vigenère cipher — • Hill cipher — • Rotor machines (1870–1943) —	
3.3	Cifrari a flusso (Stream cipher) Operazione di base: XOR —	
3.4	One-Time Pad Sicurezza del cifrario OTP —	
3.5	Pseudo-Random Generators (PRG) Tre Tipi di Generatori — • Requisiti di un buon PRNG — • PRNG deboli da non usare — • Un buon PRNG: Blum Blum Shub (BBS) — • Struttura generica e sicurezza computazionale — • RC4 — • Sicurezza di RC4: debolezze — • Stream cipher moderni: eStream (nonce-based) —	
3.6	Attacchi su OTP e Stream Cipher Attacco 1: Two-Time Pad — • Attacco 2: nessuna integrità (OTP è malleabile) —	
Chapter 4	Block Ciphers	Page
4.1	Performance dei cifrari	

- 4.2 DES
 - Storia — • Feistel Network — • Struttura a 16 round — • Lunghezza della chiave e parity bit — • Key schedule (generazione di $K_1, \dots, K_{16}$) — • Weak keys — • Initial Permutation (IP) e la sua inversa — • La funzione di round f — • Le S-box — • Avalanche effect —
- 4.3 Ricerca esaustiva su DES
 - DES challenge —
- 4.4 Rafforzare DES contro la ricerca esaustiva
 - Double DES — • Triple DES — • DESX —
- 4.5 Altri attacchi sui block cipher
 - Differential cryptanalysis (Biham-Shamir) — • Linear cryptanalysis (Matsui) — • Attacchi all'implementazione —
- 4.6 AES
 - Il processo di standardizzazione — • Caratteristiche e parametri — • Substitution-Permutation Network (SPN) — • Il processo di cifratura — • La round function — • La S-box di AES — • Sicurezza e attacchi —
- 4.7 Ricerca esaustiva: quante coppie servono?

Chapter 5

Modes of Operation _____ Page _____

- 5.1 ECB — Electronic CodeBook
 - Problema di sicurezza —
- 5.2 La nozione di sicurezza
 - Semantic security con chiave one-time (IND-EAV) — • Semantic security con chiave many-time (IND-CPA) —
- 5.3 ECB non e' semantically secure
- 5.4 Rendere sicura la cifratura
 - Soluzione 1: Randomized encryption — • Soluzione 2: Nonce-based encryption —
- 5.5 CBC — Cipher Block Chaining
 - Initialization Vector (IV) — • Attacco a CBC con IV prevedibile —
- 5.6 Da block cipher a stream cipher
 - CFB — Cipher FeedBack — • OFB — Output FeedBack — • CTR — Counter —
- 5.7 XTS-AES (storage devices)

Chapter 6

Key Exchange _____ Page _____

- 6.1 Distribuzione delle chiavi pubbliche
 - Public announcement — • Directory pubblica dinamica — • Public-key authority — • Certificati a chiave pubblica —
- 6.2 Distribuzione delle chiavi segrete
 - Key management: il problema — • Online Trusted Third Party (TTP) — • Un protocollo giocattolo (toy protocol) —
- 6.3 Key exchange senza TTP online
 - Merkle Puzzles (1974) — • Il protocollo Diffie-Hellman —

Chapter 7

Asymmetric Cryptography _____ Page _____

- 7.1 Dai cifrari simmetrici a quelli asimmetrici
- 7.2 Crittografia asimmetrica: principi
 - Confidenzialita' — • Autenticazione — • Differenze con la crittografia simmetrica —
- 7.3 Public-key encryption: definizione formale
- 7.4 Trapdoor one-way functions
 - Funzioni hash — • Uso scorretto di una TDF —

- 7.5 RSA
 - Richiami: aritmetica modulo composti — • La trapdoor permutation RSA — • Esempio numerico — • Assunzione RSA e sicurezza — • Textbook RSA e' insicuro — • RSA in pratica —
- 7.6 Firme digitali (Digital Signatures)
 - Firma basata su RSA — • Documenti lunghi: hash-then-sign — • Proprieta' —
- 7.7 Altri cryptosystem asimmetrici
 - ElGamal — • Rabin — • Elliptic Curve Cryptography (ECC) —

Chapter 8

IPsec and TLS _____ Page _____

Chapter 9

Access Control _____ Page _____

Chapter 10

Wireless Network Security _____ Page _____

- 10.1 L'ambiente wireless/radio
 - Perché il wireless è più rischioso del cablato —
- 10.2 Tassonomia delle minacce
 - Attacchi passivi vs attivi —
- 10.3 Attacchi al livello radio
 - Jamming — • Frequency Hopping Spread Spectrum (FHSS) — • Eavesdropping / Sniffing — • Replay Attack — • Rolling Code — • Challenge and Response — • Cifratura radio —
- 10.4 RFID
 - Cloning dei tag RFID — • Spoofing RFID —
- 10.5 IEEE 802.11 — Wi-Fi
 - Layer fisico — • Accesso al canale: CSMA/CA —
- 10.6 Meccanismi di "sicurezza" deboli
 - SSID Hiding — Security by Obscurity — • MAC Address Whitelisting — • Management Frames in chiaro —
- 10.7 WEP — Wired Equivalent Privacy
 - WEP — Shared Key — • WEP — Open System — • WEP — IV Collision —
- 10.8 IEEE 802.11i e WPA/WPA2
 - Modi di utilizzo — • Schemi di protezione (802.11i) — • TKIP — • CCMP — Counter Mode-CBC MAC Protocol —
- 10.9 Attacchi a WPA e WPS
 - Attacchi a WPA-PSK — • WPS (Wi-Fi Protected Setup) — • Rogue AP —
- 10.10 KRACK e WPA3
 - KRACK — Key Reinstallation AttaCK — • WPA3 —
- 10.11 Riepilogo e concetti chiave
 - Evoluzione della sicurezza Wi-Fi — • Concetti chiave — • Strumenti e riferimenti menzionati —

Chapter 11

Exploits and Patches _____ Page _____

Chapter 12

Attacchi di Buffer Overflow _____ Page _____

- 12.1 Introduzione
 - Buffer overflow attacks — • Exploits — • Storia degli incidenti più importanti —
- 12.2 Definizione di buffer overflow

- 12.3 Esecuzione di un programma
 - I compilatori — • Esempio $C \rightarrow x86-64$ — • Memoria del computer — • Memoria del programma e Instruction Pointer (IP) —
- 12.4 Lo Stack e la chiamata di funzione
 - Lo Stack — • Chiamata di una funzione — • Esempio di Stack Frame — • Esempio di Ricorsione — • Review —
- 12.5 Stack Buffer Overflow
 - Un esempio semplice di programma vulnerabile — • Cosa succede se s è più lunga di 32 caratteri? — • Crash del programma e DoS —
- 12.6 Smashing the Stack
 - Idea chiave — • Exploit v1 — layout di memoria — • Exploit v1 — sovrascrittura —
- 12.7 Shellcode
 - Caratteristiche — • Restrizioni generiche sul contenuto — • Un classico shellcode per attacco execve — • Generare shellcode —
- 12.8 NOP Sled
 - Hitting the Target — il problema — • NOP Sled — • Exploit v2 con NOP sled —
- 12.9 Funzioni C non sicure e altre forme di overflow
 - Routine comuni della libreria C non sicure — • Altre forme di attacchi di overflow —
- 12.10 Difese contro i Buffer Overflow
 - Stack Canaries — StackGuard — • Stack non eseguibile — • Address-Space Layout Randomization (ASLR) — • Write Safe Code! —

Ci sara' una domanda sul lab

Example 0.0.1

Quali opzioni ho per crackare una password?

Chapter 1

Discrete Probability (a short reminder)

Un breve ripasso di probabilità discreta: serve per ragionare sulla sicurezza dei cifrari (es. la perfect secrecy dell'OTP e l'uniformità del keystream).

1.1 Distribuzione di probabilità

Definition 1.1.1: Universe / Sample space

U è un **insieme finito**, detto *Universe* o *Sample space*.

Esempi: lancio di una moneta $U = \{\text{heads}, \text{tail}\}$ (oppure $U = \{0, 1\}$); lancio di un dado $U = \{1, 2, 3, 4, 5, 6\}$. In crittografia spesso $U = \{0, 1\}^n$.

Definition 1.1.2: Distribuzione di probabilità

Una *distribuzione di probabilità* P su U è una funzione

$$P : U \rightarrow [0, 1] \quad \text{tale che} \quad \sum_{x \in U} P(x) = 1$$

Esempi: moneta $P(\text{heads}) = P(\text{tail}) = 1/2$; dado $P(1) = \dots = P(6) = 1/6$. Oppure su $U = \{0, 1\}^2 = \{00, 01, 10, 11\}$: $P(00) = 1/2$, $P(01) = 1/8$, $P(10) = 1/4$, $P(11) = 1/8$ (somma = 1).

1.1.1 Distribuzioni particolari

1. **Distribuzione uniforme**: $\forall x \in U, P(x) = 1/|U|$.
2. **Point distribution** in x_0 : $P(x_0) = 1$ e $\forall x \neq x_0, P(x) = 0$.

... e molte altre.

1.2 Eventi

Sia U un universe con distribuzione P .

Definition 1.2.1: Evento

Un *evento* è un sottoinsieme $A \subseteq U$. La sua probabilità è

$$\Pr[A] = \sum_{x \in A} P(x)$$

Note:

Si noti che $\Pr[U] = 1$.

Example 1.2.1 (Dado)

$U = \{1, \dots, 6\}$ uniforme, $A = \{1, 3, 5\}$ (numeri dispari):

$$\Pr[A] = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{1}{2}$$

Example 1.2.2 (Bit meno significativi)

$U = \{0, 1\}^8$ con distribuzione uniforme ($P(x) = 1/2^8$ per ogni x). Sia

$$A = \{x \in U : \text{lsb}_2(x) = 11\} \subseteq U$$

(gli x i cui due bit meno significativi sono 11, cioè della forma -----11). Poiché $\Pr[A] = \frac{1}{2^8} \cdot |A|$ e $|A| = 2^6$ (i 6 bit liberi):

$$\Pr[A] = \frac{2^6}{2^8} = \frac{1}{4}$$

1.2.1 Unione di eventi

Dati due eventi A_1, A_2 , anche $A_1 \cup A_2$ è un evento e:

$$\Pr[A_1 \cup A_2] = \Pr[A_1] + \Pr[A_2] - \Pr[A_1 \cap A_2]$$

Proposition 1.2.1 Union bound

$$\Pr[A_1 \cup A_2] \leq \Pr[A_1] + \Pr[A_2]$$

Se inoltre $A_1 \cap A_2 = \emptyset$ (eventi disgiunti) allora vale l'uguaglianza:

$$A_1 \cap A_2 = \emptyset \implies \Pr[A_1 \cup A_2] = \Pr[A_1] + \Pr[A_2]$$

1.3 Variabili aleatorie**Definition 1.3.1: Variabile aleatoria**

Una *random variable* X è una funzione $X : U \rightarrow V$. X assume valori in V e *induce* una distribuzione su V .

Example 1.3.1 (Dado pari/dispari)

$U = \{1, \dots, 6\}$ uniforme, $X : U \rightarrow V = \{\text{"even"}, \text{"odd"}\}$ con

$$X(2) = X(4) = X(6) = \text{"even"}, \quad X(1) = X(3) = X(5) = \text{"odd"}$$

Allora $\Pr[X = \text{"even"}] = \frac{1}{2}$ e $\Pr[X = \text{"odd"}] = \frac{1}{2}$.

1.3.1 La variabile aleatoria uniforme

Sia S un insieme (es. $S = \{0, 1\}^n$). Scriviamo

$$X \leftarrow S$$

per indicare che X è una **variabile aleatoria uniforme** su S , cioè

$$\forall a \in S : \Pr[X = a] = \frac{1}{|S|}$$

1.3.2 Algoritmi randomizzati

- **Algoritmo deterministico:** $y \leftarrow A(m)$, a ogni input corrisponde un unico output.
- **Algoritmo randomizzato:** l'output è una variabile aleatoria $y \leftarrow A(m)$ (lo stesso input può dare output diversi, secondo una distribuzione).

1.4 Indipendenza

Definition 1.4.1: Eventi indipendenti

Due eventi A e B sono *indipendenti* se

$$\Pr[A \cap B] = \Pr[A] \cdot \Pr[B]$$

Definition 1.4.2: Variabili aleatorie indipendenti

Due variabili aleatorie X, Y a valori in V sono *indipendenti* se

$$\forall a, b \in V : \Pr[X = a \text{ and } Y = b] = \Pr[X = a] \cdot \Pr[Y = b]$$

1.5 XOR e la sua proprietà fondamentale

Lo XOR di due stringhe in $\{0, 1\}^n$ è la loro somma bit a bit modulo 2 ($\oplus$); $\text{XOR}(X)$ indica lo xor di tutti i bit della stringa X (la parità). La proprietà che lo rende utilissimo in crittografia:

Theorem 1.5.1 Proprietà fondamentale dello XOR

Siano:

1. X una variabile aleatoria su $\{0, 1\}^n$ con distribuzione **uniforme**;
2. Y una variabile aleatoria su $\{0, 1\}^n$ con distribuzione **arbitraria**;
3. X e Y **indipendenti**.

Allora $Z := Y \oplus X$ è una variabile aleatoria **uniforme** su $\{0, 1\}^n$.

Dimostrazione (per $n = 1$): Sia $\Pr[Y = 0] = p_0$ e $\Pr[Y = 1] = p_1$ (con $p_0 + p_1 = 1$), mentre X è uniforme: $\Pr[X = 0] = \Pr[X = 1] = \frac{1}{2}$. Per indipendenza, la coppia (X, Y) ha probabilità $\Pr[X = x] \cdot \Pr[Y = y]$. Allora $Z = 0$ accade quando $X = Y$:

$$\Pr[Z = 0] = \Pr[(X, Y) = (0, 0)] + \Pr[(X, Y) = (1, 1)] = \frac{p_0}{2} + \frac{p_1}{2} = \frac{p_0 + p_1}{2} = \frac{1}{2}$$

Di conseguenza $\Pr[Z = 1] = \frac{1}{2}$, quindi Z è uniforme.



Note:

È esattamente questa proprietà che dà la perfect secrecy all'OTP: facendo lo XOR del messaggio (distribuzione *qualsiasi*) con una chiave *uniforme* e indipendente, il ciphertext risulta uniforme e non rivela informazione sul plaintext.

1.6 Il paradosso del compleanno (birthday paradox)

Siano $r_1, \dots, r_n \in U$ variabili aleatorie **independent identically distributed** (i.i.d.).

Theorem 1.6.1 Birthday paradox

Quando $n = 1.2 \times |U|^{1/2}$, allora

$$\Pr[\exists i \neq j : r_i = r_j] \geq \frac{1}{2}$$

cioè con circa $\sqrt{|U|}$ campioni si ha una **collisione** con probabilità $\geq 1/2$.

Example 1.6.1 (Compleanni)

$U = \{1, 2, \dots, 366\}$. Con $n = 1.2 \times \sqrt{366} \approx 23$ persone, due hanno lo stesso compleanno con probabilità $\geq 1/2$ (sorprendentemente poche!).

Example 1.6.2 (Spazio crittografico)

$U = \{0, 1\}^{128}$. Dopo aver campionato circa 2^{64} messaggi random da U , è probabile che due coincidano. Questo è il motivo per cui un valore casuale a n bit "regge" solo fino a $\sim 2^{n/2}$ usi.

Note:

Vedremo più avanti un esempio pratico con **WEP** (Wired Equivalent Privacy, IEEE 802.11): l'IV a 24 bit collide dopo $\sim 2^{12}$ frame per il birthday paradox (in realtà si ripete del tutto dopo 2^{24}). La probabilità di collisione cresce ad "S" con il numero di campioni n e raggiunge $1/2$ attorno a $1.2\sqrt{|U|}$.

Chapter 2

Modular Arithmetic

L'aritmetica modulare e' lo strumento matematico alla base di gran parte dei cifrari (affine, Hill, RSA, Diffie-Hellman, ...). La introduciamo qui perche' serve gia' a giustificare il cifrario affine.

2.1 L'insieme $\mathbb{Z}_m$

Definition 2.1.1: Aritmetica modulo m

$\mathbb{Z}_m$ e' l'insieme $\{0, 1, \dots, m-1\}$ equipaggiato con due operazioni, $+$ e $\times$. Addizione e moltiplicazione funzionano esattamente come quelle reali, con l'unica differenza che il risultato viene **ridotto modulo m** (si tiene il resto della divisione per m).

Example 2.1.1

Calcoliamo 11×13 in $\mathbb{Z}_{16}$. Si ha $11 \times 13 = 143$. Riduciamo modulo 16:

$$143 = 8 \times 16 + 15 \implies 143 \bmod 16 = 15$$

quindi $11 \times 13 = 15$ in $\mathbb{Z}_{16}$.

2.2 Congruenza

Definition 2.2.1: Congruenza modulo n

Due interi a e b si dicono *congruenti modulo n* se $(a \bmod n) = (b \bmod n)$, e si scrive

$$a \equiv b \pmod{n}$$

Equivalentemente: la loro differenza $b - a$ e' un multiplo intero di n , ovvero a e b danno lo stesso resto quando divisi per n .

Example 2.2.1

- $73 \equiv 4 \pmod{23}$, infatti $73 = 23 \cdot 3 + 4$
- $21 \equiv -9 \pmod{10}$, infatti $21 = 3 \cdot 10 - 9$

2.3 Invertibilita' e gcd

Per poter *decifrare* un cifrario basato su una moltiplicazione modulare serve poter **invertire** l'operazione. Questo non e' sempre possibile: dipende dal gcd (massimo comun divisore, *greatest common divisor*).

Definition 2.3.1: Coprimalita'

Due interi a e m si dicono *coprime* (o *relatively prime*) se $\gcd(a, m) = 1$.

Theorem 2.3.1 Teorema 2.1 (Stinson)

La congruenza

$$ax \equiv b \pmod{m}$$

ha un'unica soluzione $x \in \mathbb{Z}_m$ per ogni $b \in \mathbb{Z}_m$ se e solo se $\gcd(a, m) = 1$.

Note:

In altre parole, la mappa $x \mapsto ax \bmod m$ e' una biiezione (e quindi invertibile) esattamente quando a e m sono coprime. Se invece $\gcd(a, m) = d > 1$ la mappa non e' iniettiva: input diversi collidono sullo stesso output, e la decifratura diventa ambigua.

Fonte: *Cryptography Theory and Practice*, 4th ed., D. R. Stinson & M. B. Paterson.

Chapter 3

Introduzione e Cifrari Simmetrici

3.1 Introduction to Cryptography

Definition 3.1.1: Cryptography

Art and science of using mathematics to obscure the meaning of data by applying transformations to the data that are impractical or impossible to reverse without the knowledge of some key

Definition 3.1.2: Cryptoanalysis

Art/science of breaking encryption without knowing the key

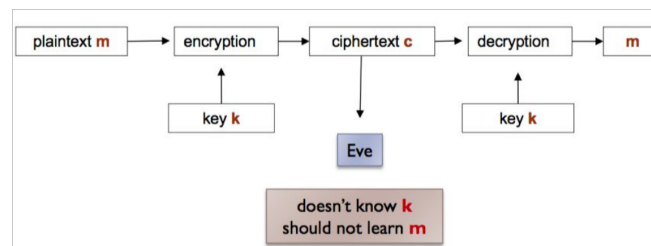
Used for:

- **Communication:** web traffic, wireless, vpn
- **Files on disk**
- **User authentication**

For secure communication, we also want to ensure no eavesdropping or tampering. Possible approaches are:

- **Steganography:** we 'hide' the existence of the message
- **Cryptography:** we instead hide the meaning of the message

3.1.1 Encryption Terminology



3.1.2 Goals and Protocols

The basic goals are:

- **Privacy**
- **Authenticity**

- **Integrity**
- **Non-repudiation:** no disclaiming of authorship (guarantees Authenticity and Integrity)

The *protocols* need to guarantee these goals by understanding:

- The parties and the context
- The goals
- The **trusted computing base**
- The capabilities of the ... (**Threat Model**)

3.1.3 Kerchoff's Principle and the Threat Model

Important rule regarding the safety of cyber systems

Theorem 3.1.1

The security of a protocol shouldn't assume that the underlying methods/algorithms of the encryption are secret, as only the secrecy of the keys can be guaranteed.

Security by obscurity does not work.

So the encryption functions need to remain secure even with the attacker knowing how the function works. The attacker threat model consists of:

- Knowledge about the cipher (Kerchoff)
- Interaction with the messages and the protocol
- Interaction with the encryption algorithm
 - **Ciphertext-only**
 - **Chosen-plaintext attack (CPA)**
 - **Chosen-ciphertext attack (CCA)**
 - CPA and CCA may be *adaptive* (previous requests may change choices)
- Available resources (storage/computation)

3.1.4 Symmetric Encryption

Lowkey l'abbiamo fatta già due volte, aggiungo solo roba che dice in più' (sta letteralmente leggendo le slide come il guerriero)

- **Single-use key:** the key is regenerated for each transmitted file
- **Multi-use key:** the key is used with a "nonce" in order to be used for multiple transmissions

3.1.5 Asymmetric Encryption

Private and public key. La pubblica è salvata in una **public repo** gestita da un'autorità trusted

3.1.6 Secure Communication

Una volta SSL ora TLS, poi RCS per end-to-end security (sono la stessa cosa??). Es. su Internet il protocollo **TLS** (Transport Layer Security, predecessore: **SSL**) fornisce data encryption, data origin authentication e message integrity. Funziona in due fasi: (1) *secret key establishment* (si sceglie/condivide una chiave segreta comune), (2) *secure communication* con confidentiality e integrity.

3.1.7 But crypto can do much more

Oltre alla comunicazione sicura, la crittografia abilita anche:

- **Secure messaging** tra mobile (es. Rich Communication Services, RCS).
- **Digital signatures**: la firma di una stessa persona *cambia* a seconda del documento (a differenza di una firma autografa); si usa crittografia asimmetrica (servono due chiavi).
- **Anonymous communication**: MIX networks, Onion routing / TOR (Bob si chiede "con chi ho appena parlato?").

Note:

Things to remember. La crittografia E': uno strumento potentissimo, base di molti meccanismi di sicurezza. La crittografia NON E': la soluzione a tutti i problemi di sicurezza; affidabile a meno che non sia implementata e usata correttamente; qualcosa che dovresti provare a inventarti da solo (esistono moltissimi esempi di design ad-hoc rotti).

3.2 Cifrari classici

Cryptography is a rigorous science that follows three main steps:

- Specify Threat Model
- Propose a construction
- ...

un po di storia, va veloce (manca la post-quantum criptology)
vabbè sto scrivendo tutot in italiano facciamolo in italiano
la brodez sta davvero ripetendo il cifrario di Cesare....

3.2.1 Cifrario di Cesare / Shift cipher

La chiave e' un numero k (per Cesare $k = 3$): per cifrare si "shifta" ogni lettera di k posizioni a destra, per decifrare si shifta indietro (a sinistra) di k . E' un **cifrario a sostituzione**.

Vista matematica. Sia $P = C = K = \mathbb{Z}_{26}$ (le 26 lettere $A, \dots, Z$ viste come $0, \dots, 25$). Per $k \in K$, $0 \leq k \leq 25$:

$$e_k(p) = p + k \bmod 26 \quad d_k(c) = c - k \bmod 26$$

Example 3.2.1 (Shift con $k = 11$)

plaintext = CRYPTOGRAPHYISFUN, $k = 11$:

- $C \rightarrow 2$; $2 + 11 \bmod 26 = 13 \rightarrow N$
- $R \rightarrow 17$; $17 + 11 \bmod 26 = 2 \rightarrow C$
- ...
- $N \rightarrow 13$; $13 + 11 \bmod 26 = 24 \rightarrow Y$

ciphertext = NCJAVZRCLASJTDQFY. (Di solito spazi e punteggiatura vengono eliminati.)

Sicurezza. Lo spazio delle chiavi e' minuscolo (≤ 26 chiavi), quindi e' banale da battere con una ricerca esaustiva (**brute force** / exhaustive-search cryptanalysis): si provano tutte le 25 chiavi ($k = 0$ e' l'identita') finche' la decifrazione "ha senso" (*makes sense*). Una volta trovato k , decifrare e' immediato.

3.2.2 Monoalphabetic substitution cipher

La chiave e' una **permutazione** π dell'alfabeto $\Sigma = \{A, B, C, \dots, Z\}$. Per cifrare, ogni lettera X del plaintext viene sostituita con $\pi(X)$; per decifrare si applica $\pi^{-1}(Y)$ a ogni lettera Y del ciphertext.

Lo spazio delle chiavi ha dimensione $|\mathcal{K}| = 26! \approx 2^{88}$: troppo grande per il brute force.

Ma: la sostituzione monoalfabetica *preserva le caratteristiche statistiche della lingua*, quindi e' vulnerabile alla **frequency analysis** (Al-Kindi, ≈ 800 d.C.). Le idee:

1. Frequenza delle singole lettere. In inglese le piu' comuni sono **e** (12.7%), **t** (9.1%), **a** (8.1%).
2. Frequenza dei **digrammi** (coppie): **th**, **he**, **in**, **er**, **an**, **re**, ...
3. Frequenza dei **trigrammi**: **the**, **ing**, **and**, ...

Si contano le occorrenze di caratteri/combinazioni nel ciphertext, si cercano pattern e ripetizioni, e si sostituiscono i caratteri cifrati con i loro probabili equivalenti in chiaro. Esempio: se nel ciphertext la lettera piu' frequente e' **B**, probabilmente $B \rightarrow E$; il digramma piu' frequente $\rightarrow$ **IN**; il trigramma piu' frequente $\rightarrow$ **THE**; ecc.

3.2.3 Affine cipher

Un altro caso speciale della sostituzione e' il **cifrario affine**, in cui la funzione di cifratura ha la forma

$$e(x) = ax + b \pmod{26}, \quad a, b \in \mathbb{Z}_{26}$$

Per $a = 1$ ci si riduce allo shift cipher.

Quando e' decifrabile? La decifratura e' possibile sse la funzione affine e' *iniettiva*: per ogni $y \in \mathbb{Z}_{26}$ vogliamo che la congruenza $ax + b \equiv y \pmod{26}$ abbia un'unica soluzione in x . Questo equivale a $ax \equiv y - b \pmod{26}$, e poiche' $y - b$ spazia su tutto $\mathbb{Z}_{26}$ al variare di y , basta studiare

$$ax \equiv y \pmod{26}$$

Per il Teorema 2.1 (vedi il capitolo *Modular Arithmetic*) questa ha soluzione unica per ogni y sse $\gcd(a, 26) = 1$ (cioe' a e 26 coprimi).

Example 3.2.2 (Chiave non valida)

Se $\gcd(a, 26) = d > 1$ il cifrario non e' valido. Es. $\gcd(4, 26) = 2$: la funzione $e(x) = 4x + 7 \pmod{26}$ NON e' una cifratura valida perche' non iniettiva. Infatti 'a' e 'n' cifrano entrambe in H (in generale x e $x + 13$ collidono):

$$\text{affinecrypt('a', 4, 7)} = \text{affinecrypt('n', 4, 7)} = \text{'H'}$$

3.2.4 Vigenere cipher

(Blaise de Vigenere, XVI sec.) E' un cifrario **polialfabetico**: risolve la debolezza principale dei monoalfabetici, in cui ogni lettera del ciphertext corrisponde sempre alla stessa (unica) lettera del plaintext.

Setup. Dato un intero positivo m , con $P = C = (\mathbb{Z}_{26})^m$ e chiave $K = (k_1, k_2, \dots, k_m)$ una *keyword* di lunghezza m :

- **Encryption:** $e_k(p_1, \dots, p_m) = (p_1 + k_1, p_2 + k_2, \dots, p_m + k_m) \pmod{26}$
- **Decryption:** $d_k(c_1, \dots, c_m) = (c_1 - k_1, c_2 - k_2, \dots, c_m - k_m) \pmod{26}$

La keyword viene ripetuta ciclicamente lungo tutto il messaggio.

Example 3.2.3 (keyword CRYPTO ($m = 6$))

k	C	R	Y	P	T	O	C	R	Y	P	T	O
m	W	H	A	T	A	N	I	C	E	D	A	Y
c	Y	Y	Y	I	T	B	K	T	C	S	T	M

(somma posizione per posizione mod 26)

Cryptanalysis (come romperlo). E' di fatto una *collezione di m shift cipher* (uno per ogni posizione modulo m). Una stessa lettera del ciphertext puo' corrispondere a piu' lettere del plaintext (es. $Y\ Y\ Y \Leftrightarrow W\ H\ A$), quindi la frequency analysis diretta e' piu' difficile. Procedura:

1. Indovinare la lunghezza m della chiave (es. **Kasiski test**, Friedrich Kasiski 1863; Charles Babbage $\approx$ 1854).
2. Dividere il ciphertext in m sotto-sequenze, ognuna cifrata con un singolo shift cipher.
3. Applicare la frequency analysis su ciascuna. (Es. se in una sotto-sequenza la lettera piu' comune e' G , probabilmente $G \rightarrow E$, quindi la lettera di chiave e' $k_i = G - E = C$, da $c_i = m_i + k_i \Rightarrow k_i = c_i - m_i$.)

3.2.5 Hill cipher

(1929, Lester S. Hill.) Generalizza la sostituzione usando **matrici**. Sia $m \geq 2$, $P = C = (\mathbb{Z}_{26})^m$ e

$$\mathcal{K} = \{\text{matrici } m \times m \text{ invertibili su } \mathbb{Z}_{26}\}$$

Per una chiave K :

$$e_K(x) = xK \quad d_K(y) = yK^{-1}$$

con tutte le operazioni in $\mathbb{Z}_{26}$. In forma estesa, il vettore riga $y = (y_1, \dots, y_m)$ si ottiene moltiplicando il vettore riga $x = (x_1, \dots, x_m)$ per la matrice $K = (k_{i,j})$.

Example 3.2.4 ($m = 2$)

$$(y_1, y_2) = (x_1, x_2) \begin{pmatrix} 11 & 8 \\ 3 & 7 \end{pmatrix} \iff \begin{cases} y_1 = (11x_1 + 8x_2) \bmod 26 \\ y_2 = (3x_1 + 7x_2) \bmod 26 \end{cases}$$

3.2.6 Rotor machines (1870–1943)

Il Vigenère si rompe una volta trovata la lunghezza della chiave. Come avere una chiave *molto* piu' lunga? Idea:

- **Multiple rounds** di sostituzione: cifrare significa mappare una lettera molte volte.
- Cablaggio meccanico/elettrico per automatizzare cifratura e decifratura.

Una macchina e' fatta di piu' cilindri (**rotori**) che rimappano le lettere piu' volte; i rotori ruotano a ogni battitura, cambiando la sostituzione a ogni carattere.

- **Hebern machine**: esempio iniziale a *singolo rotore*.
- **Enigma**: la piu' famosa, con 3–5 rotori piu' un *reflector*.

3.3 Cifrari a flusso (Stream cipher)

In Un symmetric cipher definito su (K, P, C) , con K chiave, plaintext P e ciphertext C , abbiamo che la funzione di encryption è spesso randomizzata ed è definita come $E : K \times P \rightarrow C$ e invece la funzione di decryption è deterministica e tale per cui $D : C \rightarrow P$.

3.3.1 Operazione di base: XOR

Lo XOR di due stringhe in $\{0, 1\}^n$ è la loro somma bit a bit modulo 2 ($\oplus$):

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0$$

Note:

Proprietà che useremo di continuo: $k \oplus k = 0$ e $x \oplus 0 = x$ (sono quelle alla base della correttezza dell'OTP). La notazione $\text{XOR}(X)$ può indicare lo xor di *tutti* i bit di X (la parità): es. $X = 00100011 \Rightarrow \text{XOR}(X) = 1$.

3.4 One-Time Pad

Definition 3.4.1: One-Time Pad (Gilbert S. Vernam, 1917, Bell Labs)

Si definisce *One-Time Pad* il seguente cifrario:

- Spazio delle chiavi: $0, 1^n$ (stringhe di bit di lunghezza n)
- Funzione di encryption: $E_k(m) = m \oplus k$
- Funzione di decryption: $D_k(c) = c \oplus k$

Dove k è la chiave e m è il plaintext e $\text{len}(k) = \text{len}(m)$

Praticamente si fa lo XOR tra i bit del plaintext e quelli della chiave. In questo caso la lunghezza della key deve essere uguale a quella del plaintext.

Note:

Due condizioni *cruciali* per la sicurezza: la chiave k deve essere **random** (distribuzione *uniforme* su K) e va usata **una sola volta** (*one-time*). Vedremo negli attacchi cosa succede se si violano.

Proposition 3.4.1 Sulla correttezza dell'Otp

$$D(k, E(k, m)) = k \oplus (k \oplus m) = (k \oplus k) \oplus m = 0 \oplus m = m$$

pro: è mega veloce sia in encryption che in decryption brodezz contro: deve usare chiavi lunghissime perché la key deve essere lunga come il messaggio. ma dato che questa key deve essere mandata da alice a bob in modo sicuro, allora potrebbe usare sto meccanismo sicuro per mandare direttamente il messaggio, quindi boh non serve a un cazzo.

3.4.1 Sicurezza del cifrario OTP

PER supporre che un messaggio sia sicuro, si potrebbero usare due approcci naif:

- **Tentativo #1 — l'attaccante non riesce a trovare la chiave:** Non basta. Controesempio: il cifrario $E(k, m) = m$ (che ignora la chiave) soddisferebbe questo requisito pur essendo del tutto insicuro — la chiave non si trova semplicemente perché non serve.
- **Tentativo #2 — l'attaccante non riesce a recuperare l'intero plaintext:** Non basta neanche. Controesempio: $E(k, m_0 \| m_1) = m_0 \| (k \oplus m_1)$ lascia m_0 completamente in chiaro. Se l'attaccante recupera anche solo una parte del messaggio, per molte applicazioni è già un disastro.

SI RINGRAZIA PROFONDAMENTE IL BUON CLAUDE SHANNON PER LA DEFINIZIONE DI SICUREZZA, OVVERO CHE *ciphertext non deve rivelare assolutamente nessuna informazione sul plaintext*. Zero. Nulla. Neanche quante vocali ci sono, neanche la lunghezza approssimativa. Si noti la seguente definizione:

Definition 3.4.2: Sicurezza perfetta secondo Shannon

Un cifrario (E, D) su (K, P, C) ha *perfect secrecy* se $\forall m_0, m_1 \in P : \text{len}(m_0) = \text{len}(m_1)$ e $\forall c \in C$ allora:

$$\Pr[E(k, m_0) = c] = \Pr[E(k, m_1) = c]$$

Dove k è la chiave

Note:

In altre parole la probabilità che il ciphertext c è identica qualunque sia il messaggio cifrato. Pertanto non c'è informazione in c su quale sia m_0 o m_1 . Questa definizione (Shannon, 1949) non fa *nessuna* assunzione computazionale sull'attaccante: per questo si chiama anche *unconditional security* / *perfect security* (l'avversario più potente immaginabile non impara nulla del plaintext dal ciphertext $\Rightarrow$ nessun ciphertext-only attack).

Si noti il seguente teorema:

Theorem 3.4.1

Il One-Time Pad ha perfect secrecy

Dimostrazione: Sia m e c , vogliamo calcolare $Pr[E(k, m) = c]$ dove k è uniforme su K .

Tale probabilità è uguale al numero di chiavi $k \in K$ tali che $E(k, m) = c$ diviso al numero di chiavi totali $|K|$, pertanto basta dimostrare che questa probabilità è costante $\forall(m, c)$

Nell'OTP $E(k, m) = m \oplus k$ implica $k = m \oplus c$. C'è esattamente una chiave per ogni coppia (m, c) che produce quel ciphertext. Il numeratore quindi è sempre 1, indipendentemente da m e c .

Pertanto si che

$$Pr[E(k, m) = c] = \frac{1}{|K|} \quad \forall m, c$$

La probabilità è la stessa per qualunque messaggio



Si ha tuttavia una conseguenza importante

Theorem 3.4.2 Teorema di Shannon

La sicurezza perfetta implica che $|K| \geq |P|$

Note:

Ovvero un cifrario ha sicurezza perfetta se e solo se la lunghezza della chiave è almeno quella del messaggio

Se volete sicurezza perfetta, pagate il prezzo di chiavi lunghissime. E questo è impraticabile su larga scala.

La conseguenza pratica è drastica: dobbiamo rinunciare alla sicurezza perfetta e accontentarci di qualcosa di "meno perfetto", la sicurezza computazionale, che è comunque più che sufficiente.

3.5 Pseudo-Random Generators (PRG)

Abbiamo due fatti:

- Il OTP è perfettamente sicuro ma richiede chiavi lunghe quanto il messaggio
- La sicurezza perfetta è impossibile con chiavi corte.

La soluzione è quella di rinunciare alla sicurezza perfetta e sostituiamo la chiave veramente veramente casuale con una sequenza pseudo-casuale generata da una chiave corta. Questo è l'idea degli stream cipher. Formalmente

Definition 3.5.1: Stream Cipher

Si definisce *Stream Cipher* un sistema che mira a emulare la sicurezza del One-Time Pad in modo efficiente. A differenza dei cifrari a blocchi che operano su segmenti fissi di dati, lo stream cipher elabora il testo in chiaro bit per bit o byte per byte

Adesso si noti la seguente definizione

Definition 3.5.2: Pseudo-Random Generator (PRG)

Si definisce *PRG* la seguente funzione:

$$G : \{0, 1\}^s \rightarrow \{0, 1\}^n \text{ con } n \gg s$$

In pratica riceve in input un seed (la chiave breve, s bit) e produce una sequenza molto più lunga (n bit) che sembra casuale. Lo stream cipher usa G così

- $E(k, m) = G(k) \oplus m$
- $D(k, c) = G(k) \oplus c$

La chiave k è breve (il seed), la sequenza $G(k)$ è lunga quanto il messaggio.

3.5.1 Tre Tipi di Generatori

Vale la pena distinguere tre concetti

Definition 3.5.3: TRNG (True Random Number Generator)

usa fenomeni fisici genuinamente casuali ex rumore termico, eventi radioattivi, jitter dell'orologio. Produce vera casualità, ma è lento

Definition 3.5.4: PRNG (Pseudo-Random Number Generator)

algoritmo deterministico che a partire da un seed produce una sequenza lunga e statisticamente "casuale"

Note:

Veloce ma deterministico, da un seed produce sempre la stessa sequenza

Definition 3.5.5: PRF (Pseudo-Random Function)

una funzione che, dato un input, produce un output pseudo-casuale

Note:

è una generalizzazione del PRNG

3.5.2 Requisiti di un buon PRNG

Un PRNG per uso crittografico deve soddisfare due requisiti fondamentali

- **Randomness (apparente)**: la sequenza prodotta deve superare tutti i test statistici noti — distribuzione uniforme dei bit, assenza di pattern, correlazioni basse tra bit vicini
- **Unpredictability (imprevedibilità)**: dato un prefisso della sequenza, non deve esistere nessun algoritmo efficiente (in tempo polinomiale) in grado di predire il bit successivo con probabilità significativamente maggiore di $1/2$ (**next-bit test**)

Requisiti sul seed. Dato che il PRNG è deterministico, tutta la sicurezza si scarica sul seed:

- per uso crittografico il seed deve essere **sicuro** (imprevedibile);
- il seed stesso deve essere un numero random o pseudo-random;
- tipicamente il seed è generato da un **TRNG** (a partire da una entropy source) e poi dato in pasto al PRNG.

Note:

Requisito minimo: la lunghezza s del seed deve essere **sufficientemente grande**, così che una ricerca esaustiva sui 2^s seed possibili sia infeasible per l'attaccante.

3.5.3 PRNG deboli da non usare

Due esempi di PRNG statisticamente accettabili ma crittograficamente insicuri

- **glibc random():** Utilizza un generatore a feedback lineare (non-linear additive feedback). conoscendo alcuni output si può calcolare facilmente il seed e predire tutti gli output futuri
- **Linear Congruential Generator:** Si ottiene attraverso questo algoritmo $r[i] = (a \cdot r[i-1] + b) \bmod p$. Semplice, veloce, ha buone proprietà statistiche, ma è matematicamente elementare da invertire: dati due output consecutivi, si trovano immediatamente a e b

3.5.4 Un buon PRNG: Blum Blum Shub (BBS)

Il BBS è un esempio di PRNG crittograficamente sicuro la cui sicurezza è ridotta a un problema matematico difficile

Algorithm 1: Generatore Blum Blum Shub

```
1 Scegli due grandi numeri primi  $p$  e  $q$  tali che  $p \equiv q \equiv 3 \pmod{4}$ ;  
2 Calcola  $n \leftarrow p \cdot q$ ;  
3 Scegli un seed  $s$  tale che  $\gcd(s, n) = 1$ ;  
4 Calcola  $X_0 \leftarrow s^2 \bmod n$ ;  
5 for  $i = 1$  to  $k$  do  
6    $X_i \leftarrow X_{i-1}^2 \bmod n$ ;  
7    $B_i \leftarrow X_i \bmod 2$ ;  
8 return  $\{B_1, B_2, \dots, B_k\}$ ;
```

Proposition 3.5.1 Sicurezza di BBS

Predire il bit successivo della sequenza BBS è computazionalmente equivalente a fattorizzare $n = p \cdot q$

Note:

non esiste nessun algoritmo efficiente noto per fattorizzare numeri grandi pertanto BBS è crittograficamente sicuro

Tuttavia è relativamente lento rispetto ad altri PRNG crittografici. Per applicazioni ad alta velocità si usano altri approcci.

3.5.5 Struttura generica e sicurezza computazionale

Gli Stream Cipher: Struttura Generica Uno stream cipher tipico funziona così: a partire dalla chiave K e da un valore di inizializzazione IV , si mantiene uno stato interno s_i che evolve a ogni passo. A ogni passo si produce un byte del keystream z_i tramite una funzione $g(s_i)$, e lo stato avanza con una funzione $f(s_i)$. Il keystream viene poi XOR-ato byte a byte con il plaintext: $c_i = p_i \oplus z_i$. L' IV serve a garantire che lo stesso key K produca keystream diversi in sessioni diverse — un meccanismo simile al nonce che vedremo nei cifrari moderni.

Gli Stream Cipher Non Hanno Perfect Secrecy Una domanda naturale: uno stream cipher può avere perfect secrecy? No, per definizione. La chiave k è corta (il seed), mentre il messaggio è lungo. Per il teorema di Shannon, $|K| \geq |P|$ è necessario per la perfect secrecy — condizione violata da qualsiasi stream cipher pratico. Dobbiamo quindi usare una definizione diversa di sicurezza, la sicurezza computazionale: il sistema è sicuro se non esiste nessun algoritmo efficiente in grado di distinguere il keystream $G(k)$ da una sequenza veramente casuale. La sicurezza dello stream cipher dipende interamente dalla qualità del PRG: un PRG computazionalmente indistinguibile dal random implica uno stream cipher computazionalmente sicuro.

3.5.6 RC4

Il RC4 (Rivest Cipher 4) è stato lo stream cipher dominante per decenni, usato in SSL/TLS per HTTPS e in WEP/WPA per le reti Wi-Fi. Oggi, tuttavia, è ufficialmente deprecato ed è considerato insicuro per qualsiasi applicazione crittografica.

Struttura : usa un array di stato S di 256 byte (una permutazione dei valori 0-255) e una chiave di lunghezza variabile tra 8 e 2048 bit

Fase 1 :

Algorithm 2: Inizializzazione di S

```
1 for  $i = 0$  to 255 do
2    $S[i] = i$ ;
3    $T[i] = K[i \bmod \text{keylen}]$ ;
4  $j = 0$ ;
5 for  $i = 0$  to 255 do
6    $j = (j + S[i] + T[i]) \bmod 256$ ;
7   swap( $S[i], S[j]$ );
```

Note:

Inizializza l'array di 256 byte. Prende la tua chiave segreta (che può variare da 40 a 2048 bit) e la utilizza per "mescolare" l'array

Fase 2 :

Algorithm 3: Generazione del keystream

```
1  $i = 0$ ;
2  $j = 0$ ;
3 while true do
4    $i = (i + 1) \bmod 256$ ;
5    $j = (j + S[i]) \bmod 256$ ;
6   swap( $S[i], S[j]$ );
7    $t = (S[i] + S[j]) \bmod 256$ ;
8   output  $S[t]$ ;
```

Note:

A ogni ciclo, continua a scambiare due byte all'interno dell'array ed estrae un valore pseudo-casuale in uscita

3.5.7 Sicurezza di RC4: debolezze

RC4 ha diverse debolezze note che lo hanno reso deprecato:

1. **Bias nell'output iniziale:** anche assumendo un setup *perfetto* (permutazione uniforme tra le 256! possibili), Mantin e Shamir (*A practical attack on broadcast RC4*, FSE 2001) hanno mostrato che l'output è sbilanciato: $\Pr[2^{\text{byte}} = 0] = 2/256 = 1/128$ (il doppio dell'atteso). Rimedio: scartare i primi n byte del keystream $\Rightarrow$ **RC4-drop[n]**.
2. **Bias sui digrammi** (Fluhrer e McGrew, *Statistical analysis of the alleged RC4 keystream generator*, FSE 2000): $\Pr[(0, 0)] = 1/256^2 + 1/256^3$.
3. **Related-key attacks:** alla base dell'attacco a WEP (vedi sotto).

3.5.8 Stream cipher moderni: eStream (nonce-based)

I progetti del portfolio **eStream** introducono un **nonce** nel PRG:

$$\text{PRG} : \{0, 1\}^s \times R \rightarrow \{0, 1\}^n \quad n \gg s$$

dove il primo argomento è il *seed* e il secondo è il **nonce** r .

Definition 3.5.6: Nonce

Un *nonce* è un valore che non si ripete per una data chiave: la coppia (k, r) non viene mai usata più di una volta.

Note:

Questo permette di **riusare la chiave** k finché cambia il nonce r — risolve il problema dell'OTP/stream cipher di dover cambiare chiave a ogni messaggio.

La cifratura diventa:

$$E(k, m, r) = m \oplus \text{PRG}(k, r)$$

Esempi moderni: **Salsa20** e **ChaCha20**.

3.6 Attacchi su OTP e Stream Cipher

Lo stream cipher (e l'OTP) hanno due debolezze fondamentali se usati male: **riuso della chiave** e **mancanza di integrità**.

3.6.1 Attacco 1: Two-Time Pad

Mai usare la stessa chiave dello stream cipher più di una volta! Supponiamo che Alice cifri due messaggi con la *stessa* k :

$$c_1 = m_1 \oplus \text{PRG}(k) \quad c_2 = m_2 \oplus \text{PRG}(k)$$

L'attaccante (eavesdropper) calcola:

$$c_1 \oplus c_2 = (m_1 \oplus \text{PRG}(k)) \oplus (m_2 \oplus \text{PRG}(k)) = m_1 \oplus m_2$$

Il keystream si cancella! E c'è **abbastanza ridondanza** in inglese e nella codifica ASCII che, da $m_1 \oplus m_2$, si riescono a recuperare sia m_1 che m_2 .

Note:

Venona project (1941–1980): la NSA americana riuscì a decifrare messaggi sovietici degli anni '40 proprio perché i sovietici *riusavano* le chiavi nello schema one-time pad.

Esempi reali di riuso della chiave

- **MS-PPTP** (Windows NT, Point-to-Point Tunneling Protocol): client e server usavano la *stessa* chiave per entrambe le direzioni. I messaggi client→server $[m_1||m_2||m_3]$ e server→client $[s_1||s_2||s_3]$ erano cifrati con lo stesso $\text{PRG}(k) \Rightarrow$ two-time pad. **Servono chiavi diverse per C→S e S→C.**
- **802.11b WEP** (con RC4): la chiave di frame è $\text{PRG}(IV||k)$ dove k è una **long-term key** e IV è un Initialization Vector di soli **24 bit**. Problemi:
 - l' IV si ripete dopo $2^{24} \approx 16\text{M}$ frame (e su alcune schede si resetta a 0 dopo un power cycle);
 - le chiavi per frame consecutivi $(1||k), (2||k), \dots$ sono **molto correlate** (related keys), e RC4 non è sicuro con chiavi così correlate. Attacco FMS (2001) recupera k dopo $\sim 10^6$ frame; attacchi recenti $\sim 40\,000$ frame.

Una costruzione migliore

Invece di concatenare $IV||k$, si espande $\text{PRG}(k)$ una sola volta e si ricavano le chiavi per i singoli frame da segmenti diversi dell'output $\Rightarrow$ ogni frame ha una chiave pseudo-random (non correlata). Soluzione ancora migliore: usare un metodo di cifratura più forte, come in **WPA2**.

Riassunto Two-Time Pad

Mai riusare la chiave di uno stream cipher. In pratica:

- **Traffico di rete:** si negozia una nuova chiave per ogni sessione (es. TLS), con una sotto-chiave per $C \rightarrow S$ e una per $S \rightarrow C$.
- **Disk encryption:** di solito *non* si usa uno stream cipher ma un block cipher.

3.6.2 Attacco 2: nessuna integrità (OTP è malleabile)

L'OTP (e lo stream cipher) garantisce confidenzialità ma **non** integrità: è *malleabile*. Le modifiche al ciphertext non vengono rilevate e hanno un effetto **prevedibile** sul plaintext.

Sia $c = k \oplus m$. L'attaccante intercetta c e inoltra $c^* = c \oplus p$. Bob decifra:

$$k \oplus c^* = k \oplus c \oplus p = k \oplus k \oplus m \oplus p = m \oplus p$$

Quindi l'attaccante applica al plaintext una modifica p a sua scelta, senza conoscere m né k .

Example 3.6.1 (Flip di un bit (sì/no))

Alice risponde sì (1) o no (0) a un invito, cifrando con OTP. L'attaccante non recupera la risposta, ma può **ribaltarla** applicando $\oplus 1$ al ciphertext: se $m = 0$, allora $k \oplus (c \oplus 1) = 0 \oplus 1 = 1$; se $m = 1$, diventa 0.

Example 3.6.2 (Da Alice a Maria)

Il messaggio è "From Alice". L'attaccante vuole cambiare Alice in Maria. Conoscendo (o indovinando) la posizione del campo, calcola

$$c^* = c \oplus (\text{Alice} \oplus \text{Maria})$$

e Bob decifra $D(k, c^*) = \text{Maria}$:

$$k \oplus c^* = k \oplus k \oplus \text{Alice} \oplus \text{Alice} \oplus \text{Maria} = \text{Maria}$$

Si pensi al numero di conto in un bonifico...

Chapter 4

Block Ciphers

Lavora su blocchi di lunghezza fissa, puo' generare un po' di delay.

Definition 4.0.1: Block cipher

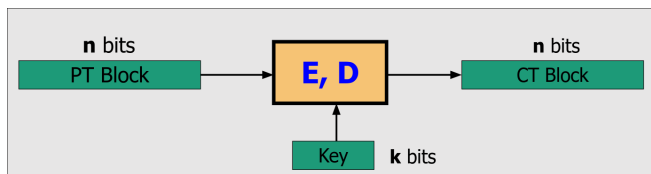
Un *block cipher* e' una coppia di funzioni (E, D) che mappano un blocco di n bit in un blocco di n bit usando una chiave di k bit:

$$E : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n, \quad D : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$$

con $D(k, E(k, m)) = m$ per ogni chiave k e ogni blocco m .

I principali sono:

- **DES**: $n = 64$ bits (8 byte), $k = 56$ bits (ma possiamo anche dire 64, vedremo perche')
- **3DES**: $n = 64$ bits, $k = 3 \cdot 56 = 168$ bits
- **AES**: $n = 128$ bits, $k = 128, 192, 256$ bits



Il block cipher e' costruito per iterazioni:

1. Key expansion: partendo da chiave lunga, genero l sottochiavi k_i (dove l e' il numero di iterazioni)
2. Applico la **round function** a m_i e alla sottochiave k_i per generare m_{i+1} (dove m_1 e' il plain text)

Definition 4.0.2: Round function

La *round function* $R(k_i, m_i) = m_{i+1}$ e' la trasformazione applicata a ogni iterazione, che combina lo stato corrente m_i con la sottochiave k_i .

Il numero di iterazioni dipende dal cifrario: **DES** ($l = 16$), **3DES** ($l = 48$) e **AES-128** ($l = 10$).

Il problema e' che e' molto lento rispetto agli altri cifrari, cosa che AES migliora abbastanza rispetto a DES.

4.1 Performance dei cifrari

Benchmark con la libreria **Crypto++ 5.6.0** (open-source, scritta da Wei Dai) su AMD Opteron 2.2 GHz (Linux):

Cipher	Block/key size	Speed (MB/sec)
RC4 (<i>stream</i>)	–	126
Salsa20/12 (<i>stream</i>)	–	643
Sosemanuk (<i>stream</i>)	–	727
3DES (<i>block</i>)	64/168	13
AES-128 (<i>block</i>)	128/128	109

Note:

Gli stream cipher sono in genere piu' veloci dei block cipher; tra i block cipher, AES e' nettamente piu' veloce di 3DES.

4.2 DES

4.2.1 Storia

- **Primi anni '70:** Horst Feistel progetta *Lucifer* all'IBM (key-length = 128 bit, block-length = 128 bit).
- **1973:** l'NBS (National Bureau of Standards, oggi NIST) richiede proposte per un block cipher; IBM presenta una variante di Lucifer.
- **1976:** l'NBS adotta **DES** come standard federale, con key-length = **56** bit e block-length = **64** bit.
- **1997:** DES viene rotto con ricerca esaustiva (*brute-force attack*) $\Rightarrow$ problema delle chiavi corte (56 bit!).
- **2000:** il NIST adotta *Rijndael* come AES, per rimpiazzare DES.

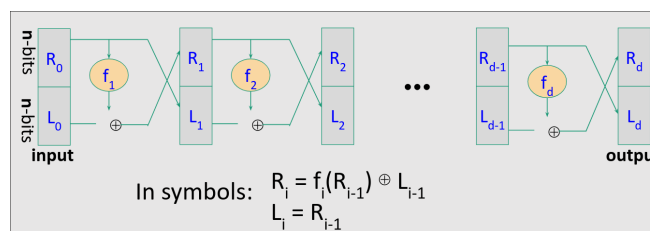
4.2.2 Feistel Network

L'idea core e' il *Feistel Network*.

Definition 4.2.1: Feistel network

Date funzioni $f_1, \dots, f_d : \{0, 1\}^n \rightarrow \{0, 1\}^n$ (**non** per forza invertibili), il *Feistel network* costruisce una funzione **invertibile** $F : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$ iterando, su input diviso in due meta' (L_{i-1}, R_{i-1}):

$$R_i = f_i(R_{i-1}) \oplus L_{i-1}, \quad L_i = R_{i-1}$$



Proposition 4.2.1

Per ogni scelta (anche **arbitraria**) di $f_1, \dots, f_d$, il Feistel network F e' **invertibile**.

Dimostrazione (per costruzione dell'inversa): Vogliamo ricavare (L_{i-1}, R_{i-1}) da (L_i, R_i) . Dalle due equazioni del round:

- da $L_i = R_{i-1}$ otteniamo subito $R_{i-1} = L_i$;
- da $R_i = f_i(R_{i-1}) \oplus L_{i-1} = f_i(L_i) \oplus L_{i-1}$ otteniamo $L_{i-1} = f_i(L_i) \oplus R_i$.

Verifica: $f_i(L_i) \oplus R_i = f_i(L_i) \oplus f_i(L_i) \oplus L_{i-1} = L_{i-1}$. L'inversa esiste **anche se le f_i non sono invertibili**: basta poterle valutare in avanti. $\square$

Note:

Il circuito di **decifratura** e' essenzialmente lo stesso, con le funzioni applicate in ordine inverso $f_d, \dots, f_1$ (WOW!). E' un metodo generale per costruire funzioni invertibili (block cipher) a partire da funzioni arbitrarie, usato in molti block cipher ... *ma non in AES*.

4.2.3 Struttura a 16 round

DES e' un Feistel network a **16 round** con $f_1, \dots, f_{16} : \{0,1\}^{32} \rightarrow \{0,1\}^{32}$ dove $f_i(x) = F(k_i, x)$ e k_i e' una **sottochiave** (una selezione permutata di bit della chiave k). Lo schema completo e':

input 64 bit $\rightarrow IP \rightarrow$ 16 round Feistel $\rightarrow IP^{-1} \rightarrow$ output 64 bit

dove IP e' la *Initial Permutation*. La chiave master e' di 56 bit, ma ogni sottochiave k_i usata nei round e' di 48 bit. Per **invertire** (decifrare) si usano le sottochiavi in ordine inverso.

4.2.4 Lunghezza della chiave e parity bit

Nella specifica DES la chiave e' lunga 64 bit (8 byte): in ogni byte l'**ottavo bit** e' un **parity-check bit**, ottenuto come XOR dei 7 bit precedenti (parita' pari/dispari). I bit utili sono quindi $8 \times 7 = 56$: ecco perche' la chiave "vera" e' di 56 bit.

4.2.5 Key schedule (generazione di $K_1, \dots, K_{16}$)

Dalla chiave a 64 bit (parity inclusi) si generano le 16 sottochiavi a 48 bit:

1. Si applica **PC-1** (*Permuted Choice 1*) che scarta gli 8 parity bit e produce due meta' da 28 bit: C_0 e D_0 .
2. Ad ogni round si fanno dei **left shift** ciclici: $C_i = LS_i(C_{i-1})$, $D_i = LS_i(D_{i-1})$.
3. Si applica **PC-2** (*Permuted Choice 2*) che da $C_i D_i$ (56 bit) seleziona la sottochiave $K_i = PC-2(C_i D_i)$ di 48 bit.

Lo shift e' di *una* posizione se $i \in \{1, 2, 9, 16\}$, di *due* posizioni altrimenti.

Note:

Le matrici PC-1 e PC-2 sono fissate dallo standard. I parity bit (cioe' i bit 8, 16, 24, 32, 40, 48, 56, 64) non compaiono in PC-1.

PC-1 (output 56 bit: 28 a sinistra, 28 a destra)

Left						
57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
Right						
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

PC-2 (output 48 bit)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

4.2.6 Weak keys

DES usa 16 sottochiavi da 48 bit generate dalla chiave master a 56 bit (64 coi parity).

Definition 4.2.2: Weak key

Una *weak key* e' una chiave che fa generare la **stessa sottochiave** in piu' di un round.

Il risultato e' una **riduzione della complessita'** e della **sicurezza** del cifrario. Le weak key si possono evitare in fase di generazione della chiave. DES ha 4 weak key (in esadecimale):

- 01010101 01010101
- FEFEFEFE FEFEFEFE
- E0E0E0E0 F1F1F1F1
- 1F1F1F1F 0E0E0E0E

4.2.7 Initial Permutation (IP) e la sua inversa

IP e IP^{-1} sono permutazioni fisse dei 64 bit. Si presentano come tabelle solo per comodita': in realta' sono un **vettore**, non una matrice (la cella j -esima indica da quale bit dell'input prendere il bit j -esimo dell'output).

IP								IP^{-1}							
58	50	42	34	26	18	10	2	40	8	48	16	56	24	64	32
60	52	44	36	28	20	12	4	39	7	47	15	55	23	63	31
62	54	46	38	30	22	14	6	38	6	46	14	54	22	62	30
64	56	48	40	32	24	16	8	37	5	45	13	53	21	61	29
57	49	41	33	25	17	9	1	36	4	44	12	52	20	60	28
59	51	43	35	27	19	11	3	35	3	43	11	51	19	59	27
61	53	45	37	29	21	13	5	34	2	42	10	50	18	58	26
63	55	47	39	31	23	15	7	33	1	41	9	49	17	57	25

4.2.8 La funzione di round f

La funzione $f_i(x) = F(k_i, x)$ prende in input x di 32 bit e la sottochiave k_i di 48 bit:

1. **Expansion** E : x (32 bit) viene espanso a 48 bit (alcuni bit vengono replicati e spostati).
2. **XOR**: si calcola $E(x) \oplus k_i$ (48 bit).
3. **S-box**: il risultato si divide in 8 gruppi da 6 bit, ciascuno passa in una **S-box** $S_j : \{0, 1\}^6 \rightarrow \{0, 1\}^4$; in output $8 \times 4 = 32$ bit.
4. **Permutation** P : i 32 bit vengono permutati dalla *permutation box* P .

Expansion Permutation (E)						Permutation Function (P)							
32	1	2	3	4	5	16	7	20	21	29	12	28	17
4	5	6	7	8	9	1	15	23	26	5	18	31	10
8	9	10	11	12	13	2	8	24	14	32	27	3	9
12	13	14	15	16	17	19	13	30	6	22	11	4	25
16	17	18	19	20	21								
20	21	22	23	24	25								
24	25	26	27	28	29								
28	29	30	31	32	1								

4.2.9 Le S-box

Ogni S-box e' una funzione $\{0, 1\}^6 \rightarrow \{0, 1\}^4$ implementata come **look-up table**. Dei 6 bit di input:

- i due bit **esterni** (primo e ultimo) selezionano la **riga** (0-3);
- i quattro bit **centrali** selezionano la **colonna** (0-15).

Example 4.2.1 (Uso di S_5)

Calcoliamo $S_5(011011)$. I bit esterni sono $0_1 = 01 = \text{riga } 1$; i bit centrali sono $\underline{1101} = 13 = \text{colonna } 13$. La tabella di S_5 in (riga 1, colonna 13) vale $9 = 1001$, quindi $S_5(011011) \rightarrow 1001$.

		S_5														
$r \backslash c$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3

Scelta delle S-box e della P-box

Sceglierle a caso darebbe un cifrario **insicuro** (key recovery dopo circa 2^{24} output). Si seguono quindi diverse regole:

- nessun bit di output deve essere vicino a una **funzione lineare** dei bit di input;
- le S-box sono mappe **4-to-1** (4 preimmagini per ogni output);
- matematicamente una S-box e' una *funzione booleana vettoriale non lineare*.

4.2.10 Avalanche effect

Definition 4.2.3: Avalanche effect

Proprieta' desiderabile di ogni algoritmo di cifratura: un **piccolo cambiamento** nel plaintext o nella chiave deve produrre un cambiamento **significativo** nel ciphertext. In particolare, cambiare un solo bit del plaintext (o della chiave) deve cambiare *molti* bit del ciphertext.

Se il cambiamento fosse piccolo, si potrebbe sfruttare per ridurre lo spazio di plaintext/chiavi da esplorare.

4.3 Ricerca esaustiva su DES

Lo spazio chiavi di DES e' 2^{56} , troppo piccolo. La tabella riporta il tempo medio per una ricerca esaustiva:

Key size	Cipher	N. chiavi	Tempo a 10^9 dec/s	Tempo a 10^{13} dec/s
56	DES	$2^{56} \approx 7.2 \times 10^{16}$	2^{55} ns = 1.125 anni	1 ora
128	AES	$2^{128} \approx 3.4 \times 10^{38}$	5.3×10^{21} anni	5.3×10^{17} anni
168	Triple DES	$2^{168} \approx 3.7 \times 10^{50}$	5.8×10^{33} anni	5.8×10^{29} anni
192	AES	$2^{192} \approx 6.3 \times 10^{57}$	9.8×10^{40} anni	9.8×10^{36} anni
256	AES	$2^{256} \approx 1.2 \times 10^{77}$	1.8×10^{60} anni	1.8×10^{56} anni
26!	Monoalfabetico	4×10^{26}	6.3×10^9 anni	6.3×10^6 anni

4.3.1 DES challenge

Dato $msg = \text{"The unknown messages are: XXXX ..."}$ diviso in blocchi da 8 byte e cifrato con la stessa chiave k , lo scopo e' trovare $k \in \{0, 1\}^{56}$ t.c. $DES(k, m_i) = c_i$ per $i = 1, 2, 3$ e decifrare $c_4, c_5, \dots$. La storia degli attacchi:

- **1997**: ricerca distribuita su Internet (progetto **DESHALL**), 78.000 volontari $\rightarrow$ **3 mesi** (96 giorni). La soluzione era: *"Strong cryptography makes the world a safer place"*.
- **1998**: macchina **EFF** (Electronic Frontiers Foundation, *Deep Crack*) $\rightarrow$ **3 giorni** (costo 250K\$).
- **1999**: ricerca combinata (Deep Crack + distributed.net) $\rightarrow$ **22 ore**.
- **2006**: **COPACOBANA** (120 FPGA) $\rightarrow$ **7 giorni** (costo 10K\$).

$\Rightarrow$ i cifrari a 56 bit **non vanno usati**: una chiave a 56 bit e' troppo corta.

4.4 Rafforzare DES contro la ricerca esaustiva

Due metodi:

- **Metodo 1:** cifrature multiple con chiavi diverse (Double DES, **Triple-DES**, ...).
- **Metodo 2:** DESX.

Sono costruzioni generali, applicabili anche ad altri block cipher.

4.4.1 Double DES

Definiamo una funzione $2E : K^2 \times M \rightarrow M$ come $2E(k_1, k_2, m) = E(k_1, E(k_2, m))$, quindi la lunghezza della chiave viene raddoppiata $2 \times 56 = 112$.

Per la decryption applichiamo le chiavi in ordine inverso:

$$D(k_2, D(k_1, m))$$

Ma c'è un problema: dato m e c , lo scopo è di trovare (k_1, k_2) t.c.:

- $E(k_1, E(k_2, m))$
- $E(k_2, m) = D(k_1, c)$

Che sono equivalenti.

Definition 4.4.1: Meet in the Middle Attack

Attacco a una cifratura composta $E(k_1, E(k_2, m))$: invece di provare tutte le $2^{2 \cdot 56}$ coppie di chiavi, si sfrutta l'uguaglianza $E(k_2, m) = D(k_1, c)$ tabulando un lato e cercando l'altro, riducendo il tempo da 2^{112} a circa 2^{57} (a costo di 2^{56} di spazio).

Questo ci dà l'apertura per il Meet in the Middle Attack, che si svolge in due step:

1. **Step 1:** creo una tabella che per ogni riga ha la tupla $(k_i, E(k_i, m))$ (sono 2^{56} entry) e la ordino sulla seconda colonna.
2. **Step 2:** per ogni $k \in \{0, 1\}^{56}$ testo se $D(k, c)$ sta nella seconda colonna. Se c'è, allora

$$E(k_i, m) = D(k, c) \implies (k_i, k) = (k_2, k_1)$$

Il costo computazionale è di

$$\underbrace{2^{56} \log(2^{56})}_{\text{build + sort}} + \underbrace{2^{56} \log(2^{56})}_{\text{search}} < 2^{63} \ll 2^{112}$$

(prima costruire e ordinare la tabella, poi cercarci dentro), mentre lo **spazio** richiesto è $\approx 2^{56}$. Quindi 2DES **non** offre 112 bit di sicurezza: è poco più forte di DES singolo. DA STUDIARE LA METTE SEMPRE STA ROBA IN ESAME!!!!

4.4.2 Triple DES

Stessa roba del double, abbiamo tre chiavi, però è un po' diverso

$$3E(k_1, k_2, k_3, m) = E(k_1, D(k_2, E(k_3, m)))$$

- key length = $3 \times 56 = 168$ bit;
- è **stra lento** (3 operazioni, 3× più lento di DES);
- se $k_1 = k_2 = k_3$ ci troviamo nel caso di DES singolo (retrocompatibilità').

Ci sta sempre il **meet in the middle attack** (versione *homework*): si spezza in $E(k_3, \cdot)$ da un lato e $D(k_2, E(k_1, \cdot))$ dall'altro, ottenendo

$$\text{Time} = \underbrace{2^{56} \log(2^{56})}_{\text{build + sort}} + \underbrace{2^{112} \log(2^{56})}_{\text{search}} < 2^{118}, \quad \text{space} \approx 2^{56}$$

quindi costo $< 2^{118}$ (comunque non poco).

4.4.3 DESX

Definiamo

$$EX(k_1, k_2, k_3, m) = k_1 \oplus E(k_2, m \oplus k_3)$$

La key-length e' $64 + 56 + 64 = 184$ bit, ma c'e' un attacco facile in tempo $2^{64+56} = 2^{120}$.

Note:

Attenzione: usare solo lo XOR esterno $k_1 \oplus E(k_2, m)$ **oppure** solo quello interno $E(k_2, m \oplus k_1)$ e' **insicuro**: il cifrario resta debole quanto E rispetto alla ricerca esaustiva. Servono **entrambi** gli XOR.

4.5 Altri attacchi sui block cipher

4.5.1 Differential cryptanalysis (Biham-Shamir)

Definition 4.5.1: Differential cryptanalysis

E' un *chosen plaintext attack*: l'attaccante conosce un certo numero di coppie (plaintext, ciphertext) e confronta lo **XOR di due plaintext** con lo **XOR dei due ciphertext** corrispondenti:

$$\Delta_P = P_1 \oplus P_2, \quad \Delta_C = C_1 \oplus C_2$$

La distribuzione dei Δ_C dato Δ_P puo' rivelare informazioni sulla chiave (certi bit). Trovati alcuni bit, si usa la *brute-force/exhaustive search* per i rimanenti.

Note:

Sorprendentemente DES **resiste** alla differential cryptanalysis: gli autori la conoscevano gia' e progettaron le S-box apposta per resistervi. Contro DES a 8 round servono 2^{38} coppie note (pochi minuti su un PC), contro i 16 round servono 2^{47} chosen plaintext. In pratica **non e' efficace** contro DES.

4.5.2 Linear cryptanalysis (Matsui)

Descritto nel 1993 da M. Matsui (Mitsubishi Electric). Invece di cercare punti isolati in cui il cifrario si comporta come qualcosa di piu' semplice, prova a *costruire un'approssimazione (lineare) del block cipher nel suo complesso*. E' uno dei due attacchi piu' usati sui block cipher (l'altro e' la differential), e si applica ai cifrari iterativi. Matsui (1993/1994) mostro' che DES si rompe con:

- 8 round: 2^{21} known plaintext;
- 16 round: 2^{43} known plaintext (40 giorni per generare le coppie + 10 giorni per trovare la chiave).

Note:

L'attacco non ha implicazioni pratiche: richiede troppe coppie. La *ricerca esaustiva* resta l'attacco piu' efficace contro DES.

4.5.3 Attacchi all'implementazione

Non attaccano l'algoritmo ma la sua realizzazione fisica:

1. **Side channel attack**: si misura il **tempo** o la **potenza** consumata durante enc/dec (es. su una smartcard si vedono i 16 round nel grafico della corrente).
2. **Fault attack**: errori indotti nel calcolo dell'ultimo round espongono la chiave segreta k .

⇒ **non implementare da soli le primitive crittografiche.**

4.6 AES

DES non e' troppo efficiente e ha la chiave troppo corta: serve un rimpiazzo.

4.6.1 Il processo di standardizzazione

- **1997:** il **NIST** pubblica una request for proposals. Obiettivo: rimpiazzare DES per cifratura sia governativa che privata; l'algoritmo deve essere un block cipher a chiave simmetrica con block size ≥ 128 bit e key size 128, 192, 256 bit.
- **1998:** il NIST seleziona 15 candidati.
- **1999:** il NIST sceglie 5 finalisti.
- **2000:** il NIST sceglie **Rijndael** come AES (progettato in Belgio da Joan Daemen e Vincent Rijmen). Key size 128, 192, 256 bit; block size 128 bit.

AES e' diventato il cifrario simmetrico piu' usato.

4.6.2 Caratteristiche e parametri

- Non e' un cifrario di Feistel.
- Progettato per essere efficiente sia in hardware che in software.
- E' un block cipher *iterativo*: block size 128 bit (anche 192/256), key length 128, 192, 256 bit, numero di round 10, 12 o 14 (dipende dalla lunghezza della chiave), key scheduling con 44, 52 o 60 sottochiavi da 32 bit (4 byte).

Con $Nb = 4$ word (1 word = 32 bit) per il blocco:

	Key length (Nk)	Block size (Nb)	N. round (Nr)
AES-128	4	4	10
AES-192	6	4	12
AES-256	8	4	14

Note:

Lo spazio chiavi e' enorme: $2^{128} \approx 3.4 \times 10^{38}$ chiavi (un PC che prova 2^{55} chiavi/sec impiegherebbe ≈ 149.000 miliardi di anni), $2^{192} \approx 6.2 \times 10^{57}$, $2^{256} \approx 1.1 \times 10^{77}$. Probabilmente AES restera' sicuro per almeno 20 anni.

4.6.3 Substitution-Permutation Network (SPN)

A differenza di DES (Feistel), AES e' una **Substitution-Permutation Network**: ogni round e' fatto da un **substitution layer** (le S-box) seguito da un **permutation layer**, con uno XOR della sottochiave. Per decifrare bisogna **invertire ogni step** (non basta riusare lo stesso circuito come in Feistel).

4.6.4 Il processo di cifratura

Lo **State** e' una matrice quadrata 4×4 di byte (16 byte = 128 bit). La chiave (16 byte) viene espansa in 44 word. Lo schema:

1. trasformazione iniziale: **AddRoundKey** con la round-0 key;
2. $Nr - 1$ round "pieni", ciascuno con 4 trasformazioni (SubBytes, ShiftRows, MixColumns, AddRoundKey);
3. ultimo round: SubBytes, ShiftRows, AddRoundKey (**senza MixColumns**).

4.6.5 La round function

Ogni round (tranne l'ultimo) e' una composizione uniforme e parallela di 4 step:

- **SubBytes** (ByteSub): sostituzione byte-per-byte tramite una S-box, ossia $A[i, j] = S[A[i, j]]$. E' una tabella di 256 byte (facilmente calcolabile).
- **ShiftRows**: permutazione che fa uno shift ciclico delle ultime tre righe dello State (riga 0 ferma, riga 1 shift di 1, riga 2 di 2, riga 3 di 3, verso sinistra).

- **MixColumns**: sostituzione che opera sulle colonne usando l'aritmetica del campo di Galois $GF(2^8)$.
- **AddRoundKey**: XOR bit-a-bit con una porzione della chiave espansa.

4.6.6 La S-box di AES

La S-box di AES mappa 1 byte in 1 byte: i 4 bit più a sinistra del byte danno la **riga**, i 4 più a destra la **colonna**; riga e colonna indicizzano la tabella per ottenere l'output a 8 bit. Esiste anche la **inverse S-box** per decifrare.

Example 4.6.1 (Sostituzione del byte {95})

Il valore esadecimale {95} indicizza riga 9, colonna 5 della S-box, che contiene {2A}. Quindi l'input {95} viene mappato nell'output {2A}.

AES S-box (valori in esadecimale)																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

4.6.7 Sicurezza e attacchi

AES non ha attacchi pratici noti. I migliori risultati teorici sono:

- **best key recovery**: solo 4 volte meglio della ricerca esaustiva [Bogdanov, Khovratovich, Rechberger, 2011];
- **related key attack** su AES-256 [Biryukov, Khovratovich, 2009]: date 2^{99} coppie input/output da 4 chiavi correlate, si recuperano le chiavi in tempo $\approx 2^{99}$.

4.7 Ricerca esaustiva: quante coppie servono?

Goal: date poche coppie input/output $(m_i, c_i = E(k, m_i))$, trovare la chiave k . Ma con poche coppie, la chiave trovata è *unica*?

Theorem 4.7.1 Lemma (DES come ideal cipher)

Supponiamo che DES sia un *ideal cipher* (2^{56} funzioni invertibili casuali $\Pi_1, \dots, \Pi_{2^{56}} : \{0, 1\}^{64} \rightarrow \{0, 1\}^{64}$). Allora per ogni coppia (m, c) esiste **al massimo una** chiave k tale che $c = DES(k, m)$, con probabilità $\geq 1 - \frac{1}{2^{56}} \approx 99.5\%$.

Dimostrazione:

$$\Pr[\exists k' \neq k : c = DES(k, m) = DES(k', m)] \leq \sum_{k' \in \{0, 1\}^{56}} \Pr[DES(k, m) = DES(k', m)] \leq 2^{56} \cdot \frac{1}{2^{64}} = \frac{1}{2^8} = \frac{1}{256}$$

Q

Con **due** coppie DES la probabilità di unicità sale a $\approx 1 - \frac{1}{2^{71}}$; per AES-128, con due coppie, $\approx 1 - \frac{1}{2^{128}}$.

Note:

⇒ **due** coppie input/output bastano per la ricerca esaustiva della chiave.

Chapter 5

Modes of Operation

Un block cipher sa cifrare un **solo** blocco da n bit.

Definition 5.0.1: Mode of operation

Una *mode of operation* e' una regola che descrive come applicare ripetutamente un block cipher per cifrare messaggi piu' lunghi di un singolo blocco da n bit.

Le cinque modalita' principali (NIST) sono:

Mode	Descrizione	Applicazione tipica
ECB (Electronic CodeBook)	Ogni blocco e' cifrato indipendentemente con la stessa chiave.	Trasmissione sicura di valori singoli (es. una chiave).
CBC (Cipher Block Chaining)	L'input del cifrario e' lo XOR del blocco di plaintext col ciphertext precedente.	Trasmissione general-purpose a blocchi; autenticazione.
CFB (Cipher FeedBack)	Si processano s bit alla volta; il ciphertext precedente entra nel cifrario per produrre output pseudo-random, XORato col plaintext.	Trasmissione general-purpose stream-oriented; autenticazione.
OFB (Output FeedBack)	Come CFB, ma in input al cifrario va l'output di cifratura precedente; si usano blocchi pieni.	Stream su canale rumoroso (es. comunicazione satellitare).
CTR (CounTeR)	Ogni blocco di plaintext e' XORato con un contatore cifrato ; il contatore si incrementa a ogni blocco.	General-purpose a blocchi; utile per high-speed.

5.1 ECB — Electronic CodeBook

Definition 5.1.1: ECB (Electronic CodeBook)

Il messaggio viene spezzato in N blocchi **indipendenti** $(P_1, P_2, \dots, P_N)$; la stessa chiave K cifra/decifra e ogni blocco e' cifrato **separatamente**:

$$C_i = E(K, P_i), \quad P_i = D(K, C_i)$$

5.1.1 Problema di sicurezza

ECB e' **deterministico**: lo stesso blocco produce sempre lo stesso ciphertext.

$$b_i = b_j \implies c_i = c_j$$

Quindi rivela i **pattern** quando i dati si ripetono (stesso problema del cifrario di Vigenère).

Note:

L'esempio classico e' l'immagine del **pinguino Tux**: cifrata in ECB se ne riconosce ancora la sagoma (i blocchi uguali restano uguali), mentre con una modalita' randomizzata il ciphertext sembra rumore. **Non usare ECB in pratica.**

5.2 La nozione di sicurezza

La domanda "ECB e' sicuro?" richiede una definizione formale di sicurezza.

5.2.1 Semantic security con chiave one-time (IND-EAV)

Ciphertext indistinguishability sotto un attaccante **eavesdropping** (IND-EAV). Threat model: *Ciphertext-only Attack*, la chiave e' usata una sola volta e l'attaccante osserva solo il ciphertext. Si definisce un **attack game** tra un challenger (Charlie) e un avversario A :

1. Charlie sceglie k e l'algoritmo E .
2. A sceglie due messaggi $m_0, m_1 \in M$ con $|m_0| = |m_1|$.
3. Charlie sceglie $b \xleftarrow{R} \{0, 1\}$ a caso.
4. Charlie invia $c = E(k, m_b)$.
5. A indovina $b' \in \{0, 1\}$; **vince** se $b' = b$.

Per $b = 0, 1$ si definiscono gli esperimenti $EXP(0)$ ed $EXP(1)$ (in cui Charlie cifra rispettivamente m_0 ed m_1). Il **vantaggio** dell'avversario e' la differenza assoluta tra le due probabilita' che A produca $b' = 1$:

$$\text{Adv}_{SS}[A, Q] := |\Pr[EXP(0) = 1] - \Pr[EXP(1) = 1]|$$

Definition 5.2.1: Semantic security

Un cifrario $Q = (E, D)$ e' **semantically secure** se per ogni avversario "efficiente" A il vantaggio $\text{Adv}_{SS}[A, Q]$ e' "trascurabile" (*negligible*).

Example 5.2.1 (Cifrario non semantically secure)

Supponiamo esista un A efficiente che dal ciphertext deduce sempre il **LSB** (bit meno significativo) del plaintext. Costruiamo B che sceglie m_0 con $LSB(m_0) = 0$ e m_1 con $LSB(m_1) = 1$, gira A su c e ne restituisce l'output. Allora in $EXP(0)$ esce sempre 0 e in $EXP(1)$ sempre 1:

$$\text{Adv}_{SS}[B, Q] = |0 - 1| = 1$$

che non e' trascurabile $\Rightarrow Q$ **non** e' semantically secure.

5.2.2 Semantic security con chiave many-time (IND-CPA)

La semantic security e' equivalente alla *ciphertext indistinguishability under chosen plaintext attack* (IND-CPA). Qui la chiave e' usata **piu' di una volta**, quindi l'avversario vede molti ciphertext con la stessa chiave e in piu' puo' farsi cifrare messaggi a sua scelta (potere **CPA**, modellazione conservativa della realta'). L'attack game e' come prima, ma con dei round extra in cui A puo' chiedere a Charlie la cifratura di messaggi arbitrari (*query: encrypt $m \rightarrow \text{reply: } c$*) sia prima che dopo aver ricevuto il challenge. Obiettivo dell'avversario: rompere la semantic security.

5.3 ECB non e' semantically secure

Theorem 5.3.1

ECB **non** e' semantically secure per messaggi che contengono **piu' di un blocco** (known-plaintext attack).

Example 5.3.1 (Attacco a ECB)

L'avversario sceglie due messaggi da due blocchi ciascuno:

$$m_0 = \text{"Hello World"}, \quad m_1 = \text{"Hello Hello"}$$

Riceve $c = (c_1, c_2) = E(k, m_b)$ e applica la regola: *se $c_1 = c_2$ output 1, altrimenti output 0*. Per m_1 i due blocchi sono uguali $\Rightarrow c_1 = c_2$, per m_0 no. Quindi

$$\text{Adv}_{SS}[A, \text{ECB}] = 1$$

$\Rightarrow$ **non usare ECB**; la cifratura deve essere **randomizzata**.

5.4 Rendere sicura la cifratura

5.4.1 Soluzione 1: Randomized encryption

$E(k, m)$ diventa un **algoritmo randomizzato**: cifrare lo stesso messaggio due volte da' ciphertext **diversi** (con alta probabilita'). Conseguenza: il ciphertext deve essere **piu' lungo** del plaintext,

$$\text{CT-size} \approx \text{PT-size} + (\# \text{ bit random})$$

5.4.2 Soluzione 2: Nonce-based encryption

Si introduce un **nonce** n (gia' visto per gli stream cipher eStream): $E(k, m, n) = c$ e $D(k, c, n) = m$. Secondo il *NIST SP 800-90A Rev.1* un nonce serve a costruire un seed e dare un "cuscinetto" di sicurezza; deve avere almeno `security_strength/2` bit di entropia (oppure ripetersi non piu' spesso di una stringa random di quella lunghezza), deve essere **unico** al modulo crittografico ma **non** deve essere segreto (resta pero' un parametro critico). Puo' essere composto da:

- un **valore random** generato ex novo ogni volta;
- un **timestamp** di risoluzione sufficiente;
- un **sequence number** monotono crescente;
- una **combinazione** di timestamp + sequence number.

Note:

Punti chiave: la coppia (k, n) non e' **mai** usata piu' di una volta, e il nonce **non** deve essere segreto ne' random (basta che non si ripeta per la stessa chiave). Vedi anche la definizione di *nonce* data per gli stream cipher.

5.5 CBC — Cipher Block Chaining

Grazie al meccanismo di **concatenamento**, CBC e' adatto a messaggi piu' lunghi di n bit ($n = 64$ per DES, 128 per AES) e oltre alla *confidentiality* puo' servire per l'*autenticazione*. Il ciphertext C_2 dipende da C_1 , C_3 da C_2 , ecc.

Definition 5.5.1: CBC (Cipher Block Chaining)

Ogni blocco di plaintext viene messo in XOR col ciphertext precedente prima di essere cifrato; il primo usa un *Initialization Vector* IV .

$$\text{(cifratura)} \quad C_1 = E(K, P_1 \oplus IV), \quad C_j = E(K, P_j \oplus C_{j-1}) \quad j = 2, \dots, N$$

$$\text{(decifratura)} \quad P_1 = D(K, C_1) \oplus IV, \quad P_j = D(K, C_j) \oplus C_{j-1} \quad j = 2, \dots, N$$

5.5.1 Initialization Vector (IV)

L'IV deve essere **noto** sia al mittente che al ricevente ma **imprevedibile** da terzi, e va protetto da modifiche non autorizzate (es. si invia l'IV cifrato in ECB). Due metodi per generarlo:

1. generare un blocco **random** con un RNG ($IV \in \{0,1\}^n$);
2. applicare la cifratura (con la stessa chiave) a un **nonce** unico a ogni esecuzione (il nonce puo' essere un counter, un timestamp o un message number).

Note:

Nelle librerie reali l'IV e' fornito dall'utente: es. in OpenSSL `AES_cbc_encrypt(..., unsigned char *ivec, ...)`. Se l'IV non e' random va cifrato prima dell'uso, altrimenti **niente sicurezza CPA**.

5.5.2 Attacco a CBC con IV prevedibile

Theorem 5.5.1

CBC in cui l'avversario puo' **predire** l'IV **non** e' CPA-secure.

Idea: dato $c \leftarrow E_{CBC}(k, m)$, l'avversario predice l' IV^* del *prossimo* messaggio. Sceglie $0 \in X$, ottiene $c_0 = [IV, E(k, 0 \oplus IV)]$, poi sottopone $m_0 = IV^* \oplus IV$ (e un $m_1 \neq m_0$). Riceve $c = [IV^*, E(k, IV)]$ oppure $[IV^*, E(k, m_1 \oplus IV^*)]$: se $c[1] = c_0[1]$ deduce $b = 0$, altrimenti $b = 1$, vincendo il gioco.

Note:

E' esattamente il **bug di SSL/TLS 1.0**: l'IV del record $\#i$ era l'ultimo blocco di ciphertext del record $\#(i-1)$, quindi predicibile (attacco **BEAST**). L'IV deve essere imprevedibile da terzi.

5.6 Da block cipher a stream cipher

Per AES, DES o qualunque block cipher la cifratura avviene su blocchi da n bit. E' pero' possibile "convertire" un block cipher in uno **stream cipher** con tre modalita': **CFB**, **OFB**, **CTR**. Vantaggi: niente **padding** a multipli del blocco e funzionamento in **real time** (ogni carattere puo' essere cifrato e trasmesso subito).

5.6.1 CFB — Cipher FeedBack

Si processano s bit alla volta tramite uno **shift register** di b bit, inizializzato con l'IV. Siano $MSB_s(X)$ e $LSB_s(X)$ i s bit piu'/meno significativi di X :

$$\begin{aligned} I_1 &= IV \\ I_j &= LSB_{b-s}(I_{j-1}) \parallel C_{j-1} \quad j = 2, \dots, N \\ O_j &= E(K, I_j) \quad j = 1, \dots, N \\ C_j &= P_j \oplus MSB_s(O_j) \quad j = 1, \dots, N \end{aligned}$$

In decifratura cambia solo l'ultima riga: $P_j = C_j \oplus MSB_s(O_j)$ (si usa sempre la funzione di **cifratura** E , mai D).

5.6.2 OFB — Output FeedBack

Come CFB, ma in feedback va l'**output** del cifrario (non il ciphertext); l'IV deve essere un **nonce** (unico a ogni esecuzione). L'ultimo blocco e' lungo $u < b$ bit.

$$\begin{aligned} I_1 &= Nonce, \quad I_j = O_{j-1} \quad (j \geq 2), \quad O_j = E(K, I_j) \\ C_j &= P_j \oplus O_j \quad (j = 1, \dots, N-1), \quad C_N^* = P_N^* \oplus MSB_u(O_N) \end{aligned}$$

Note:

Vantaggio: gli errori di bit in trasmissione **non si propagano** (ogni C_j dipende solo dal keystream O_j , non dai ciphertext precedenti). OFB ha la struttura di un tipico stream cipher.

5.6.3 CTR — Counter

Ogni blocco e' XORato con un **contatore cifrato**. I contatori $T_1, \dots, T_N$ devono essere **diversi** per ogni blocco (tipicamente si inizializza e si incrementa di 1 modulo 2^b):

$$C_j = P_j \oplus E(K, T_j) \quad (j = 1, \dots, N-1), \quad C_N^* = P_N^* \oplus MSB_u(E(K, T_N))$$

In decifratura $P_j = C_j \oplus E(K, T_j)$.

Note:

CTR e' **parallelizzabile** (i blocchi sono indipendenti) e quindi adatto ad applicazioni *high-speed*; usa solo la funzione E .

5.7 XTS-AES (storage devices)

Nel 2010 il NIST ha approvato un'ulteriore modalita', **XTS-AES**, anche standard **IEEE 1619-2007** (IEEE Security in Storage Working Group, P1619). Descrive la cifratura di dati su dispositivi **sector-based** (dischi), dove il threat model prevede che l'avversario possa **accedere ai dati memorizzati**. Ha avuto ampio supporto industriale. Per cifrare un singolo blocco usa **due istanze** di AES con **due chiavi** Key_1 e Key_2 : una cifra l'indice del settore i (combinato con α^i) producendo un tweak T , che viene XORato col plaintext prima e dopo la seconda cifratura con Key_1 .

Chapter 6

Key Exchange

Definition 6.0.1: Key exchange

Il *key exchange* e' una procedura con cui due parti A e B cooperano per ottenere una **chiave crittografica condivisa** (segreta).

Perche' la cifratura **simmetrica** funzioni, le due parti devono condividere la stessa chiave, che va protetta dall'accesso altrui; inoltre e' opportuno **cambiare spesso** la chiave, per limitare la quantita' di dati compromessi se un attaccante la scopre. Il problema si divide in due:

- **distribuzione di chiavi pubbliche;**
- **distribuzione di chiavi segrete** (symmetric key exchange), tramite un Trusted Third Party oppure tramite crittografia asimmetrica (Merkle Puzzles, Diffie-Hellman).

6.1 Distribuzione delle chiavi pubbliche

Una chiave pubblica PU_x va resa nota agli altri, ma in modo **autenticato** (deve essere garantito che appartenga davvero a x). Quattro approcci, dal piu' debole al piu' robusto.

6.1.1 Public announcement

Ognuno annuncia (broadcast) la propria chiave pubblica.

Note:

Debolezza: chiunque puo' *forgiare* un annuncio. Un utente puo' **fingersi A** e diffondere una chiave pubblica al posto suo. Non c'e' autenticazione dell'identita'.

6.1.2 Directory pubblica dinamica

Un'entita' fidata / autorita' centrale (CA) mantiene e distribuisce una directory pubblica delle chiavi. La registrazione di PU_x deve avvenire **di persona** o tramite un canale autenticato sicuro.

Note:

Resta vulnerabile: se l'avversario riesce a **ottenere/calcolare la chiave privata della directory authority**, puo' impersonare chiunque.

6.1.3 Public-key authority

Si assume che una CA mantenga una directory dinamica delle chiavi pubbliche di tutti i partecipanti e che ognuno conosca in modo affidabile la chiave pubblica dell'autorita' PU_{auth} . Il protocollo (con timestamp T_i e nonce N_i) e':

1. $A \rightarrow auth: Request \parallel T_1$
2. $auth \rightarrow A: E(PR_{auth}, [PU_b \parallel Request \parallel T_1])$
3. $A \rightarrow B: E(PU_b, [ID_A \parallel N_1])$
4. $B \rightarrow auth: Request \parallel T_2$
5. $auth \rightarrow B: E(PR_{auth}, [PU_a \parallel Request \parallel T_2])$
6. $B \rightarrow A: E(PU_a, [N_1 \parallel N_2])$
7. $A \rightarrow B: E(PU_b, N_2)$

L'autorita' risponde cifrando con la **propria chiave privata** PR_{auth} (cosi' chiunque puo' verificarne l'origine con PU_{auth}). I nonce N_1, N_2 identificano univocamente la transazione tra A e B .

Note:

Svantaggi: l'autorita' e' un *collo di bottiglia* (va interpellata per ogni chiave di ogni altro utente) e la directory di nomi/chiavi e' *vulnerabile a manomissioni*.

6.1.4 Certificati a chiave pubblica

Definition 6.1.1: Certificato a chiave pubblica

Un *certificato* consiste di una **chiave pubblica**, un **identificatore** del proprietario della chiave, e l'intero blocco **firmato da una terza parte fidata** (tipicamente una *Certificate Authority*, CA). Permette di distribuire la chiave senza interpellare l'autorita' a ogni scambio.

Per il partecipante A l'autorita' emette un certificato della forma

$$C_A = E(PR_{auth}, [T \parallel ID_A \parallel PU_a])$$

(T timestamp, ID_A identificatore di A). A puo' poi passarlo a chiunque, che lo legge e verifica con la chiave pubblica dell'autorita':

$$D(PU_{auth}, C_A) = D(PU_{auth}, E(PR_{auth}, [T \parallel ID_A \parallel PU_a])) = (T \parallel ID_A \parallel PU_a)$$

Un certificato deve soddisfare:

- chiunque puo' **leggerlo** per ricavare nome e chiave pubblica del proprietario;
- chiunque puo' **verificare** che proviene dalla CA e non e' contraffatto;
- **solo** la CA puo' crearlo e aggiornarlo;
- chiunque puo' verificarne la **validita' temporale** (*Not before ... Not after*).

Note:

Lo schema universalmente adottato per il formato dei certificati e' lo standard **X.509**, usato nella maggior parte delle applicazioni di sicurezza di rete (IPsec, TLS). La firma del certificato si basa sulle *firme digitali* (hash del certificato non firmato, firmato con PR_{auth} ; si verifica con PU_{auth}), che vedremo con la crittografia a chiave pubblica.

6.2 Distribuzione delle chiavi segrete

Per due parti A e B , la distribuzione di una chiave segreta puo' avvenire in vari modi:

1. A genera la chiave e la consegna fisicamente a B ;
2. una terza parte sceglie la chiave e la consegna fisicamente ad A e B ;
3. se A e B hanno usato di recente una chiave, una parte trasmette la nuova chiave all'altra cifrandola con la vecchia chiave;
4. se A e B hanno ciascuno una connessione cifrata con una terza parte C , allora C puo' consegnare la chiave sui link cifrati.

6.2.1 Key management: il problema

Con n utenti, memorizzare le chiavi segrete reciproche e' difficile: servono $O(n)$ chiavi per utente e

$$\frac{n(n-1)}{2} = O(n^2) \text{ chiavi in totale}$$

(ancora di piu' a livello di applicazione o processo).

6.2.2 Online Trusted Third Party (TTP)

Soluzione migliore: una **terza parte fidata online** (TTP). Ogni utente U_i ricorda **una sola** chiave k_i , condivisa con la TTP. Quando U_1 e U_3 vogliono comunicare, la TTP fornisce loro una **session key** k_{13} (temporanea, dura quanto la connessione logica, es. una connessione TCP). Ogni utente condivide con la TTP una **master key** a lunga durata.

Note:

Esistono varianti dello schema (*Key Translation*, *Key Distribution*, con o senza *forwarding*). Nella *Key Distribution with Forwarding*: $A \rightarrow KDC : Request$; $KDC \rightarrow A : E(K_{ma}, K_s), E(K_{mb}, K_s)$; $A \rightarrow B : E(K_{mb}, K_s)$; poi sessione con K_s .

6.2.3 Un protocollo giocattolo (toy protocol)

Alice (k_A) vuole una chiave condivisa con Bob (k_B); entrambi condividono la propria master key con la TTP:

1. $Alice \rightarrow TTP$: "Alice vuole una chiave con Bob";
2. la TTP sceglie una chiave random k_{AB} e invia ad Alice

$$E(k_A, "Alice, Bob" \parallel k_{AB}); \quad ticket = E(k_B, "Alice, Bob" \parallel k_{AB})$$

3. $Alice \rightarrow Bob$: inoltra il *ticket*;
4. ora Alice e Bob condividono k_{AB} .

Se (E, D) e' un cifrario **CPA-secure**, l'eavesdropper vede $E(k_A, \dots)$ e $E(k_B, \dots)$ ma **non impara nulla** su k_{AB} .

Note:

La TTP serve **a ogni** key exchange e **conosce tutte** le session key k_{AB} . Esempio d'uso: **Kerberos** (RFC 4120), un server di autenticazione centralizzato che autentica utenti verso server e viceversa, basato *esclusivamente* su cifratura simmetrica (nessuna chiave pubblica).

Note:

Insicuro contro attacchi attivi (es. *replay attack*): l'attaccante registra una sessione tra Alice e il merchant Bob (es. un ordine di un libro) e la **ripete** a Bob, che crede che Alice stia ordinando un'altra copia.

6.3 Key exchange senza TTP online

Domanda chiave: si possono generare chiavi condivise **senza** una TTP online? **Si'!** E' il punto di partenza della crittografia a chiave pubblica: Merkle (1974), Diffie-Hellman (1976), RSA (1977). L'obiettivo: Alice e Bob vogliono una chiave condivisa, ignota all'eavesdropper, con sicurezza **solo contro l'intercettazione passiva** (no tampering, no message injection).

6.3.1 Merkle Puzzles (1974)

Risposta: *si', ma molto inefficiente*. E' una costruzione iniziale per accordarsi su un segreto condiviso anche senza segreti in comune. Strumento principale: i **puzzle**, problemi risolvibili con "un certo sforzo".

Definition 6.3.1: Puzzle

Dato un cifrario simmetrico $E(k, m)$ con $k \in \{0, 1\}^{128}$, un *puzzle* e' $E(P, \text{"message"})$ dove la chiave e' $P = 0^{96} \| b_1 \dots b_{32}$ (solo 32 bit incogniti). Si "risolve" trovando P provando tutte le 2^{32} possibilita'.

Protocollo:

- **Alice** prepara 2^{32} puzzle: per $i = 1, \dots, 2^{32}$ sceglie $P_i \in \{0, 1\}^{32}$ random e $x_i, k_i \in \{0, 1\}^{128}$ random (con gli x_i distinti), e pone

$$\text{puzzle}_i \leftarrow E(0^{96} \| P_i, \text{"Puzzle \#"} \| x_i \| k_i)$$
 poi invia tutti i puzzle a Bob.
- **Bob** sceglie un **puzzle a caso** e lo risolve per brute-force; ottiene (x_j, k_j) , usa k_j come segreto condiviso e invia x_j ad Alice.
- **Alice** cerca il puzzle con numero x_j e usa k_j come segreto condiviso.

Note:

Costi: lavoro di Alice $O(n) = O(2^{32})$, lavoro di Bob $O(n) = O(2^{32})$, lavoro dell'eavesdropper $O(n^2) = O(2^{64})$ (deve risolvere $\approx 2^{32}$ puzzle, perche' non sa quale ha scelto Bob). Il divario e' solo **quadratico**: poca sicurezza, ecco perche' e' inefficiente.

6.3.2 Il protocollo Diffie-Hellman

Idea: Alice e Bob **non** condividono alcun segreto in anticipo; si scambiano messaggi e alla fine concordano una chiave segreta k , ignota all'eavesdropper.

Logaritmo discreto

Definition 6.3.2: Radice primitiva

In aritmetica modulare, a e' una *radice primitiva* modulo p se ogni g coprimo con p e' congruo a una potenza di a modulo p . Per un primo p , le potenze di una radice primitiva a , cioe' $a \bmod p, a^2 \bmod p, \dots, a^{p-1} \bmod p$, sono distinte e sono una permutazione degli interi da 1 a $p - 1$.

Definition 6.3.3: Logaritmo discreto

Per ogni intero b e una radice primitiva a del primo p esiste un unico esponente i con

$$b \equiv a^i \pmod{p}, \quad 0 \leq i \leq p - 1$$

L'esponente i e' il *logaritmo discreto* di b in base a , modulo p .

Il protocollo

Si fissano pubblicamente un grande primo p (almeno 2048 bit, ≈ 600 cifre) e un intero $g \in \{2, \dots, p - 2\}$.

- **Alice** sceglie un segreto random $a \in \{1, \dots, p - 2\}$ e invia $g^a \bmod p$;
- **Bob** sceglie un segreto random $b \in \{1, \dots, p - 2\}$ e invia $g^b \bmod p$;
- Alice calcola $(g^b)^a \bmod p$, Bob calcola $(g^a)^b \bmod p$.

Theorem 6.3.1 Chiave condivisa di Diffie-Hellman

Alice e Bob calcolano lo stesso valore

$$(g^b)^a \equiv g^{ab} \equiv (g^a)^b \pmod{p}$$

che diventa la loro **chiave segreta condivisa** $g^{ab} \bmod p$.

Sicurezza

L'eavesdropper vede $p, g, g^a \bmod p, g^b \bmod p$: per ottenere la chiave dovrebbe calcolare $g^{ab} \bmod p$ (problema di Diffie-Hellman), legato al **problema del logaritmo discreto**.

Definition 6.3.4: Problema del logaritmo discreto (DLP)

Dati g, p e $g^k \bmod p$, trovare k . E' ritenuto **computazionalmente difficile** per p grande.

Note:

Se p e' lungo n bit (almeno $n = 2048$ per sicurezza), il miglior algoritmo noto (*General Number Field Sieve*, GNFS) ha tempo di esecuzione **sub-esponenziale** in n : $\exp(\tilde{O}(\sqrt[3]{n}))$.

Insicurezza contro Man-in-the-Middle (MiTM)

DH e' sicuro solo contro attacchi **passivi**. Contro un attacco **attivo** (MiTM) e' insicuro: l'attaccante si interpone, sceglie segreti propri a', b' e

- verso Bob inoltra $A' = g^{a'} \bmod p$ al posto di $A = g^a \bmod p$;
- verso Alice inoltra $B' = g^{b'} \bmod p$ al posto di $B = g^b \bmod p$.

Risultato: Alice e l'attaccante condividono $g^{a'b'}$, l'attaccante e Bob condividono $g^{a'b}$. L'attaccante **rilancia il traffico** leggendolo in chiaro e puo' anche **alterarlo**. Serve autenticazione (es. certificati) per difendersi.

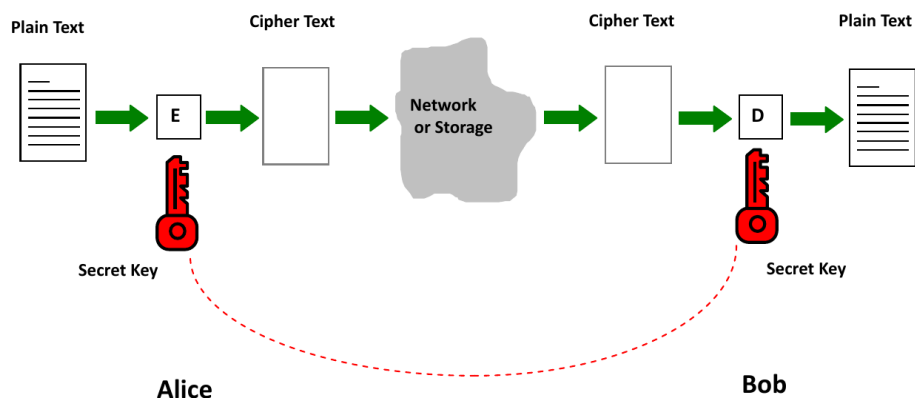
Chapter 7

Asymmetric Cryptography

7.1 Dai cifrari simmetrici a quelli asimmetrici

In un cifrario **simmetrico** (3DES, AES) Alice e Bob usano la **stessa** chiave segreta per cifrare e decifrare.

Symmetric Cipher



3

Problemi:

- per comunicare in modo sicuro devono già **condividere una chiave** (e se non si sono mai incontrati?);
- Alice deve tenere M chiavi diverse per comunicare con M persone diverse.

Note:

Da qui la domanda: due parti **senza** chiave segreta condivisa possono comunicare in modo sicuro? La risposta porta alla **crittografia asimmetrica**.

7.2 Crittografia asimmetrica: principi

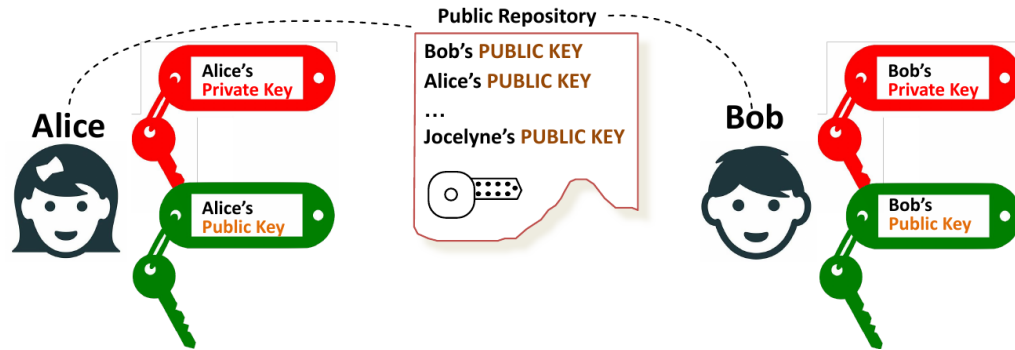
Definition 7.2.1: Crittografia asimmetrica

Ogni utente genera una **coppia di chiavi**: una viene messa in un registro pubblico (la **chiave pubblica**, PU), l'altra resta segreta (la **chiave privata**, PR). Ogni utente mantiene inoltre una collezione di chiavi pubbliche altrui.

Asymmetric Cryptography

The essential steps are the following:

- Each user generates a **pair of keys** to be used for the encryption and decryption of messages.
- Each user places one of the two keys in a public register or other accessible file. This is the **public key**.
- The companion key is kept private (the **private key**).
- Each user maintains a collection of public keys obtained from others.



6

Le due chiavi sono diverse ma **collegate**, e a seconda di quale si usa per cifrare si ottengono due scenari:

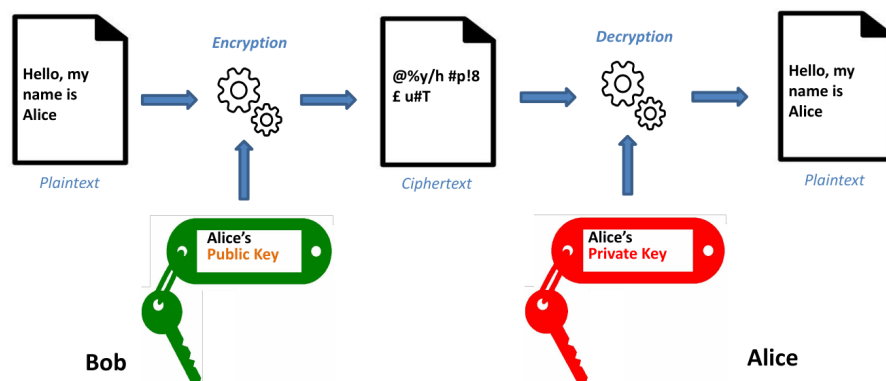
$$E(\text{private-key}_{\text{Alice}}, m) = c, \quad D(\text{public-key}_{\text{Alice}}, c) = m \quad (\text{autenticazione})$$

$$E(\text{public-key}_{\text{Alice}}, m) = c, \quad D(\text{private-key}_{\text{Alice}}, c) = m \quad (\text{confidenzialita'})$$

7.2.1 Confidenzialita'

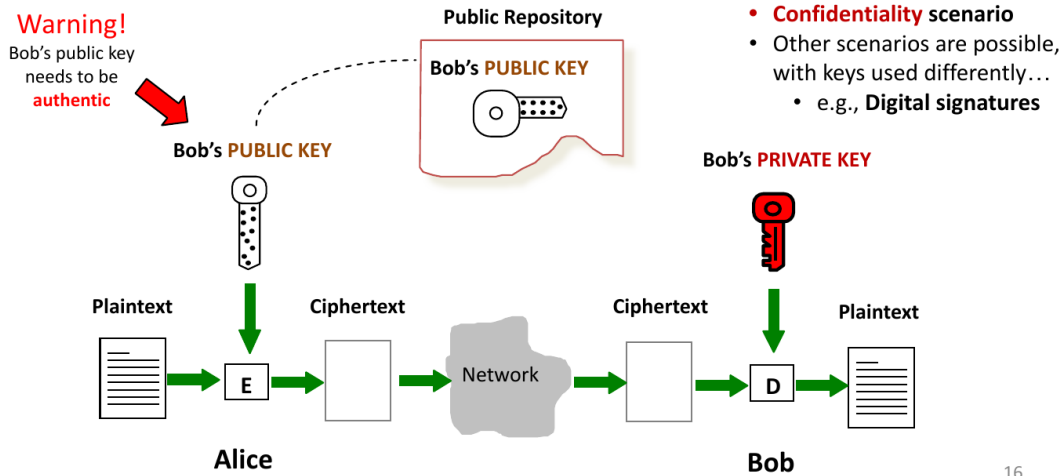
Alice cifra con la **chiave pubblica di Bob**; solo Bob, con la sua **chiave privata**, puo' decifrare.

Public and private keys confidentiality



9

Asymmetric Encryption



16

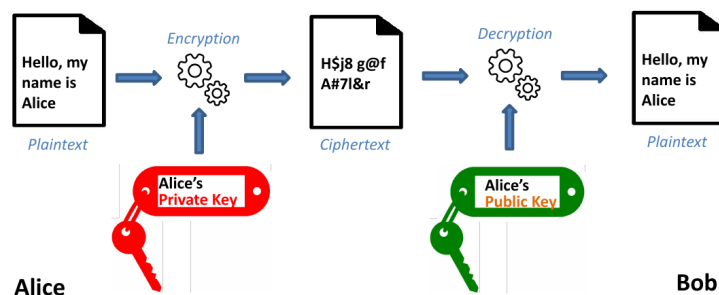
Note:

Warning: la chiave pubblica di Bob deve essere **autentica** (e' il problema risolto dai certificati, visto nel capitolo *Key Exchange*).

7.2.2 Autenticazione

Alice cifra con la **propria chiave privata**; chiunque puo' decifrare con la sua chiave pubblica, ottenendo la garanzia che il messaggio viene davvero da Alice (e' la base delle firme digitali).

Public and private keys Authentication



8

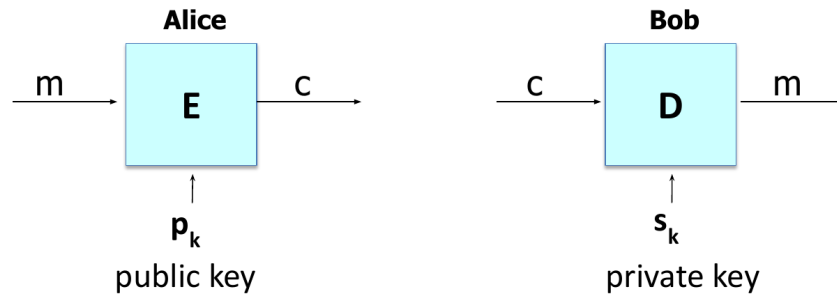
7.2.3 Differenze con la crittografia simmetrica

- la chiave pubblica e' diversa (ma collegata) dalla privata;
- e' **infeasible** per un attaccante ricavare la privata dalla pubblica;
- **non** serve distribuire una chiave segreta condivisa in anticipo;
- serve **una sola** coppia di chiavi per ogni utente (non piu' $O(n^2)$ chiavi).

7.3 Public-key encryption: definizione formale

Public key encryption

Bob: generates (p_k, s_k) and gives p_k to Alice



19

Definition 7.3.1: Sistema di cifratura a chiave pubblica

E' una tripla di algoritmi (G, E, D) :

- $G()$: algoritmo **randomizzato** che produce una coppia di chiavi (p_k, s_k) ;
- $E(p_k, m)$: algoritmo **randomizzato** che prende $m \in M$ e produce un ciphertext $c \in C$;
- $D(s_k, c)$: algoritmo **deterministico** che prende $c \in C$ e produce $m \in M$.

Consistenza: per ogni (p_k, s_k) prodotta da G e per ogni $m \in M$: $D(s_k, E(p_k, m)) = m$.

7.4 Trapdoor one-way functions

Definition 7.4.1: Trapdoor one-way function

Una funzione *facile* da calcolare in una direzione e *infeasible* nell'altra, a meno di conoscere un'informazione aggiuntiva (una "chiave", la **trapdoor**); con quell'informazione l'inversa si calcola in tempo polinomiale. E' una famiglia di funzioni invertibili f_k tali che:

$$\begin{aligned} Y &= f_k(X) && \text{facile, se } k, X \text{ noti} \\ X &= f_k^{-1}(Y) && \text{facile, se } k, Y \text{ noti} \\ X &= f_k^{-1}(Y) && \text{infeasible, se } Y \text{ noto ma } k \text{ ignoto} \end{aligned}$$

Formalmente una trapdoor function $X \rightarrow Y$ e' una tripla (G, F, F^{-1}) : $G()$ produce (p_k, s_k) ; $F(p_k, \cdot) : X \rightarrow Y$; $F^{-1}(s_k, \cdot) : Y \rightarrow X$ inverte F , con $F^{-1}(s_k, F(p_k, x)) = x$.

Definition 7.4.2: Secure TDF

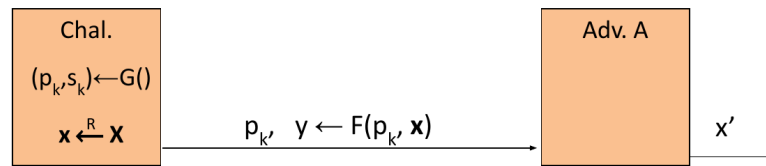
(G, F, F^{-1}) e' **sicura** se $F(p_k, \cdot)$ e' *one-way*: si puo' valutare ma non invertire senza s_k . Cioe' per ogni avversario efficiente A :

$$\text{Adv}_{\text{OW}}[A, F] = \Pr[x = x'] < \text{negligible}$$

Secure Trapdoor Functions (TDFs)

(G, F, F^{-1}) is secure if $F(p_k, \cdot)$ is a “one-way” function:

can be evaluated, but cannot be inverted without s_k



Def: (G, F, F^{-1}) is a secure TDF if for all efficient A :

$$\text{Adv}_{\text{OW}}[A, F] = \Pr[x = x'] < \text{negligible}$$

22

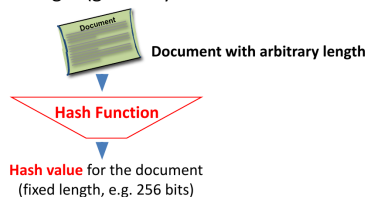
7.4.1 Funzioni hash

Definition 7.4.3: Funzione hash one-way

Una funzione $H(\cdot)$ che prende un input m di lunghezza arbitraria e produce un output $H(m)$ di lunghezza fissa L (es. MD5 $L = 128$, SHA-1 $L = 160$, SHA-256 $L = 256$ bit).

Hash Functions

- **Hash functions:**
 - **Input:** arbitrary length
 - **Output:** fixed length (generally much shorter than the input)



23

Proprietà di una buona hash:

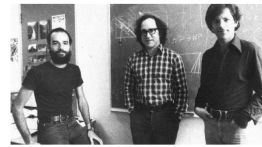
1. **Easy to evaluate:** veloce da calcolare;
2. **Hard to reverse:** dato h , infeasible trovare m con $H(m) = h$ (one-way / pre-image resistance);
3. **Hard to find collisions:** infeasible trovare $m_1 \neq m_2$ con $H(m_1) = H(m_2)$;
4. un piccolo cambiamento nel messaggio cambia l'hash in modo **estensivo** (effetto valanga).

7.4.2 Uso scorretto di una TDF

Note:

Mai cifrare applicando F direttamente al plaintext: $E(p_k, m) = F(p_k, m)$. Sarebbe **deterministico** $\Rightarrow$ non semantically secure, e soggetto a molti attacchi. La TDF va usata dentro uno schema randomizzato.

7.5 RSA



The RSA Algorithm

- One of the first practical responses to the challenge posed by Diffie-Hellman was developed by **Ron Rivest**, **Adi Shamir**, and **Len Adleman** of MIT in 1977
- Resulting algorithm is known as **RSA**
- Based on properties of **prime numbers** and results from **number theory**

28

Una delle prime risposte pratiche alla sfida di Diffie-Hellman, sviluppata da **Ron Rivest**, **Adi Shamir** e **Len Adleman** (MIT, 1977), basata su proprietà dei **numeri primi** e sulla **teoria dei numeri**. Primo annuncio: *Scientific American*, agosto 1977. Usatissima: SSL/TLS (certificati e key-exchange), e-mail e file system sicuri.

7.5.1 Richiami: aritmetica modulo composti

Sia $N = p \cdot q$ con p, q primi, $p, q \approx N^{1/2}$. $\mathbb{Z}_N = \{0, 1, \dots, N-1\}$ e $\mathbb{Z}_N^*$ e' l'insieme degli elementi invertibili.

Theorem 7.5.1 Fatti

- $x \in \mathbb{Z}_N$ e' invertibile $\iff \gcd(x, N) = 1$;
- $|\mathbb{Z}_N^*| = \phi(N) = (p-1)(q-1) = N - p - q + 1$ (funzione di Eulero);
- **Teorema di Eulero:** $\forall x \in \mathbb{Z}_N^* : x^{\phi(N)} = 1$ in $\mathbb{Z}_N$.

7.5.2 La trapdoor permutation RSA

Definition 7.5.1: RSA

$G()$: scegli due primi random $p, q \approx 1024$ bit, poni $N = pq$; scegli e, d (con $1 < e < \phi(N)$) tali che $e \cdot d \equiv 1 \pmod{\phi(N)}$. Output: chiave pubblica $p_k = (N, e)$, chiave privata $s_k = (N, d)$.

$$F(p_k, x) : \mathbb{Z}_N^* \rightarrow \mathbb{Z}_N^*, \quad y = \text{RSA}(x) = x^e \pmod{N}$$

Theorem 7.5.2 Correttezza

$F^{-1}(s_k, y) = y^d$ inverte F :

$$y^d = (x^e)^d = x^{ed} = x^{k\phi(N)+1} = (x^{\phi(N)})^k \cdot x = 1^k \cdot x = x$$

(si e' usato $ed = k\phi(N) + 1$ e il teorema di Eulero).

7.5.3 Esempio numerico

Example 7.5.1 (RSA con numeri piccoli)

Bob (generazione chiavi): $p = 5, q = 11 \Rightarrow n = 55, \phi(n) = 4 \cdot 10 = 40$; sceglie $e = 3$ ($\gcd(3, 40) = 1$) e calcola $d = 27$ ($3 \cdot 27 = 81 \equiv 1 \pmod{40}$). Chiave pubblica $(3, 55)$, privata 27.

Alice (cifratura) di $m = 13$: $c = m^e \pmod{n} = 13^3 \pmod{55} = 2197 \pmod{55} = 52$.

Bob (decifratura): $m = c^d \pmod{n} = 52^{27} \pmod{55} = 13$.

7.5.4 Assunzione RSA e sicurezza

Definition 7.5.2: RSA assumption

RSA e' una permutazione one-way: per ogni algoritmo efficiente A

$$\Pr [A(N, e, y) = y^{1/e}] < \text{negligible}$$

con p, q primi random a n bit, $N = pq$, y random in $\mathbb{Z}_N^*$.

Per invertire RSA senza d l'attaccante deve calcolare x da $c = x^e \bmod N$, cioe' le **radici e -esime** modulo N . Il miglior algoritmo noto: (1) **fattorizzare** N (*difficile!*), (2) calcolare le radici e -esime modulo p e q (*facile*). La sicurezza di RSA poggia quindi sulla difficolta' della **fattorizzazione**.

7.5.5 Textbook RSA e' insicuro

Note:

Il *textbook RSA* (cifra $c \leftarrow m^e$, decifra $c^d \rightarrow m$) e' **insicuro**: deterministico $\Rightarrow$ non semantically secure, e con molti attacchi noti. **La trapdoor permutation RSA non e' uno schema di cifratura!**

A simple attack on textbook RSA



Suppose k is 64 bits: $k \in \{0, \dots, 2^{64}\}$. Eve sees: $c = k^e$ in $\mathbb{Z}_N$

If $k = k_1 \cdot k_2$ where $k_1, k_2 < 2^{34}$ (prob. $\approx 20\%$) then $c/k_1^e = k_2^e$ in $\mathbb{Z}_N$

Meet-in-the-middle attack:

Step 1: build table: $c/1^e, c/2^e, c/3^e, \dots, c/2^{34e}$. time: 2^{34}

Step 2: for $k_2 = 0, \dots, 2^{34}$ test if k_2^e is in table. time: 2^{34}

Output matching (k_1, k_2) . Total attack time: $\approx 2^{40} \ll 2^{64}$

39

Example 7.5.2 (Attacco a textbook RSA (meet-in-the-middle))

In un handshake, il browser manda $c = \text{RSA}(k) = k^e$ con session key k a 64 bit. Se $k = k_1 \cdot k_2$ con $k_1, k_2 < 2^{34}$ (probabile $\approx 20\%$), allora $c/k_1^e = k_2^e$ in $\mathbb{Z}_N$. Si costruisce la tabella $c/1^e, c/2^e, \dots, c/2^{34e}$ (tempo 2^{34}) e poi per $k_2 = 0, \dots, 2^{34}$ si testa se k_2^e e' in tabella. Tempo totale $\approx 2^{40} \ll 2^{64}$.

Uso corretto (ISO): $E(p_k, m)$: scegli x random in $\mathbb{Z}_N$, calcola $y \leftarrow \text{RSA}(x) = x^e$ e $k \leftarrow H(x)$; output $(y, E_s(k, m))$ con (E_s, D_s) cifrario simmetrico autenticato. $D(s_k, (y, c))$: ricava $k = H(\text{RSA}^{-1}(y))$ e decifra c .

7.5.6 RSA in pratica

- **Esponente pubblico piccolo** (per cifrare in fretta): valore minimo $e = 3$ (perche' non 2? $\phi(N)$ e' pari, quindi $\gcd(2, \phi(N)) \neq 1$); valore raccomandato $e = 65537 = 2^{16} + 1$ ($\rightarrow 17$ moltiplicazioni). RSA e' **asimmetrico**: cifratura veloce, decifratura lenta.
- **Mai** usare un d piccolo per velocizzare la decifratura: Wiener (1987) $d < N^{0.25} \Rightarrow$ insicuro; Boneh-Durfee (1998) $d < N^{0.292} \Rightarrow$ insicuro (la privata si ricava da (N, e)).

- **Lunghezze chiavi** (sicurezza comparabile al simmetrico):

Cipher key-size	RSA modulus N
80 bit	1024 bit
128 bit	3072 bit
256 bit (AES)	15360 bit

Note:

Attacchi all'implementazione: timing attack [Kocher 1996], power attack [Kocher 1999], fault attack [Pellegrini 2010] (tutti possono esporre d mentre si calcola $c^d \bmod N$). Difesa comune: controllare l'output ($\approx 10\%$ di rallentamento).

Note:

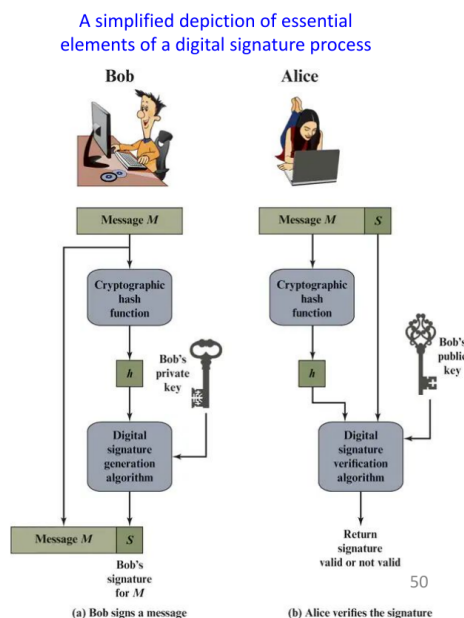
RSA Key Generation Trouble [Heninger et al. / Lenstra et al.]: con poca entropia all'avvio, piu' dispositivi generano lo **stesso** p (ma q diversi); allora per due chiavi N_1, N_2 si ha $\gcd(N_1, N_2) = p$, e si fattorizza! L'esperimento ha fattorizzato lo 0.4% delle chiavi HTTPS pubbliche. Lezione: assicurarsi che l'RNG sia ben inizializzato.

7.6 Firme digitali (Digital Signatures)

Si usa lo scenario di **autenticazione**: si firma con la chiave **privata**, si verifica con la **pubblica**.

Digital Signature

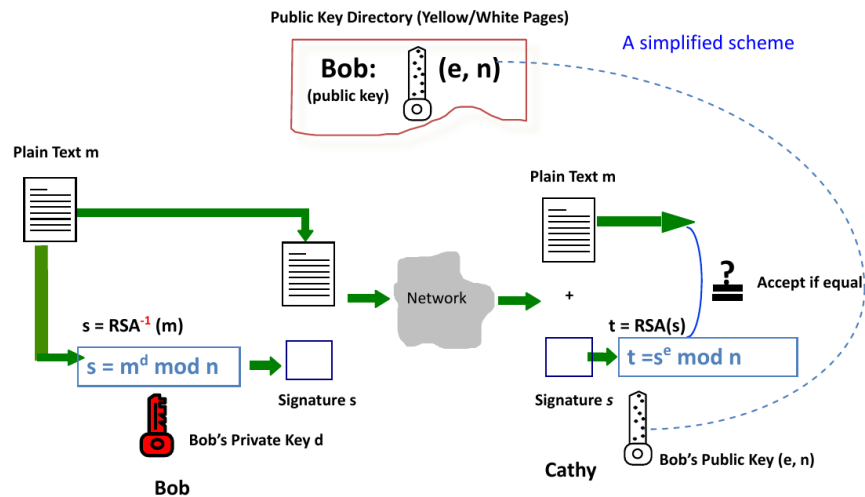
- Bob wants to send a message M to Alice.
- Although it is not important that the message M be kept secret, Bob wants Alice to be certain that message M is indeed from him.
- Bob uses a secure hash function (e.g., SHA-512) to generate a hash value h for the message M .
- The hash value h , together with Bob's private key serves as input to a digital signature generation algorithm, which produces a short block (i.e., 160 or 256 bit-binary string) that functions as a **digital signature** S .
- Bob sends message M with the signature S attached.



Bob vuole che Alice sia certa che il messaggio M viene da lui: calcola un hash $h = H(M)$ (es. SHA-512), e da h + la sua chiave privata genera la firma S ; invia (M, S) . Alice verifica con la chiave pubblica di Bob.

7.6.1 Firma basata su RSA

Digital Signature (based on *RSA*)



51

Bob firma m con la propria privata d : $s = RSA^{-1}(m) = m^d \bmod n$. Un verificatore (Cathy) calcola $t = RSA(s) = s^e \bmod n$ con la pubblica di Bob e **accetta se** $t = m$.

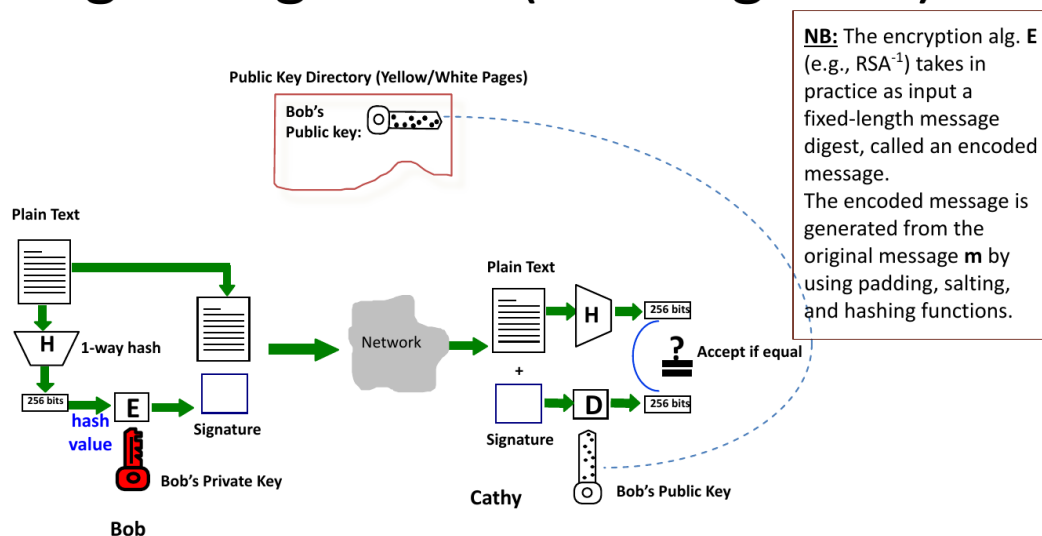
Example 7.6.1 (Firma RSA con numeri piccoli)

Chiavi come prima ($n = 55, e = 3, d = 27$). Bob firma $m = 19$: $s = 19^{27} \bmod 55 = 24$; invia $(m, s) = (19, 24)$. Cathy verifica: $t = 24^3 \bmod 55 = 19 = m \Rightarrow$ firma genuina.

7.6.2 Documenti lunghi: hash-then-sign

Un documento deve essere un intero in $[0, n)$. Per firmare documenti lunghi si fa **prima l'hash** e si firma l'hash (corto):

Digital Signature (for long docs)



57

7.6.3 Proprieta'

Una firma digitale e':

- **unforgeable** (non falsificabile, "ci vorrebbe un miliardo di anni");
- **undeniable** dal firmatario (non ripudio);
- **universalmente verificabile** (il verificatore non ha bisogno di chiavi proprie, ne' pubbliche ne' private);
- **diversa da documento a documento**.

7.7 Altri cryptosystem asimmetrici

7.7.1 ElGamal

Schema di cifratura costruito sul protocollo Diffie-Hellman.

- **Key generation** (Bob): sceglie un primo p e g radice primitiva modulo p ; un esponente random $a \in [0, p-2]$; calcola $A = g^a \bmod p$. Pubblica (p, g, A) , privata a .
- **Cifratura** di m (Alice): sceglie $b \in [0, p-2]$ random; calcola $B = g^b \bmod p$ e $c = A^b m \bmod p$; ciphertext (B, c) .
- **Decifratura** (Bob): calcola $x = p-1-a$ e $m = B^x c \bmod p$.

Note:

E' **randomizzato**; svantaggio: **message expansion** (il ciphertext e' lungo il doppio del plaintext). Sicurezza legata alla difficulta' del logaritmo discreto.

7.7.2 Rabin

- **Key generation** (Bob): due primi grandi distinti p, q con $p \equiv q \equiv 3 \pmod{4}$ (rende la decifratura piu' efficiente); $n = pq$. Pubblica n , privata (p, q) .
- **Cifratura**: $c = m^2 \bmod n$.
- **Decifratura** (via CRT): $m_p = c^{(p+1)/4} \bmod p$, $m_q = c^{(q+1)/4} \bmod q$; si trovano y_p, y_q con $y_p p + y_q q = 1$ (Euclide); $r = (y_p p m_q + y_q q m_p) \bmod n$ e $s = (y_p p m_q - y_q q m_p) \bmod n$; uno tra $r, -r, s, -s$ e' il messaggio m .

Note:

Cifratura piu' efficiente di RSA. Vantaggio teorico: recuperare m da (c, n) e' **provabilmente equivalente a fattorizzare** (non noto per RSA). Svantaggio: la decifratura da' 4 candidati.

7.7.3 Elliptic Curve Cryptography (ECC)

Crittografia basata sulle curve ellittiche: a parita' di sicurezza usa chiavi molto piu' corte di RSA. La tabella NIST SP 800-57 confronta le dimensioni delle chiavi a parita' di sforzo computazionale per la crittanalisi:

Comparable Key Sizes in Terms of Computational Effort for Cryptanalysis (NIST SP-800-57)

Symmetric Key Algorithms	Diffie-Hellman, Digital Signature Algorithm	RSA (size of n in bits)	ECC (modulus size in bits)
80	$L = 1024$ $N = 160$	1024	160–223
112	$L = 2048$ $N = 224$	2048	224–255
128	$L = 3072$ $N = 256$	3072	256–383
192	$L = 7680$ $N = 384$	7680	384–511
256	$L = 15,360$ $N = 512$	15,360	512+

Note: L = size of public key, N = size of private key.

Chapter 8

IPsec and TLS

Chapter 9

Access Control

Chapter 10

Wireless Network Security

10.1 L'ambiente wireless/radio

Per capire la sicurezza delle reti wireless bisogna prima capire l'ambiente in cui operano. Una rete wireless è costituita da **tre componenti** fondamentali, ognuno dei quali rappresenta un potenziale punto di attacco:

- **Client wireless:** il dispositivo dell'utente finale, come telefono, laptop, tablet, sensore IoT, dispositivo Bluetooth
- **Access Point (AP):** l'infrastruttura che fornisce connettività (base station cellulare, hotspot Wi-Fi, router domestico, ecc.)
- **Mezzo trasmissivo:** l'aria stessa, ovvero le onde radio che trasportano i dati tra client e AP

10.1.1 Perché il wireless è più rischioso del cablato

Ci sono quattro ragioni strutturali per cui le reti wireless sono intrinsecamente più vulnerabili:

- **Canale broadcast:** nelle reti cablate un segnale viaggia su un cavo dedicato, e per intercettarlo serve accesso fisico. In una rete wireless il segnale si propaga in tutte le direzioni: chiunque sia nel raggio può ascoltarlo. L'*eavesdropping* diventa banale.
- **Mobilità:** i dispositivi cambiano rete, si riautenticano, si spostano. Ogni cambio di rete è una finestra di vulnerabilità.
- **Risorse limitate:** sensori IoT, smartwatch, embedded device hanno CPU e memoria ridotte. Difficile implementare crittografia pesante $\Rightarrow$ più vulnerabili a DoS e malware.
- **Accessibilità fisica:** un sensore incustodito in un campo agricolo o un magazzino può essere manomesso o ascoltato senza essere notato.

10.2 Tassonomia delle minacce

Prima di entrare nel tecnico, è utile una mappa delle principali categorie di minaccia.

Definition 10.2.1: Rogue Access Point

Un attaccante configura un dispositivo wireless affinché si comporti come un AP legittimo. Gli utenti si connettono credendo di essere sulla rete autentica, esponendo credenziali e traffico.

Definition 10.2.2: Reti ad hoc

Reti peer-to-peer tra dispositivi wireless *senza AP centrale*. La mancanza di un punto di controllo rende ogni nodo costretto a fidarsi direttamente degli altri.

Definition 10.2.3: MAC Spoofing

Ogni scheda di rete ha un indirizzo MAC a 48 bit. In teoria univoco e fisso, in pratica modificabile via software in pochi secondi. Basta sniffare il traffico, individuare un MAC autorizzato e impersonarlo.

Definition 10.2.4: Man-in-the-Middle (MitM)

L'attaccante si posiziona tra client e AP. Il client crede di parlare con l'AP, l'AP crede di parlare col client: in realtà tutto passa per l'attaccante.

Definition 10.2.5: Denial of Service (DoS)

Saturazione del canale o delle risorse dell'AP con messaggi continui, impedendo il funzionamento per gli utenti legittimi. Il mezzo radio è particolarmente vulnerabile dato che è fisicamente condiviso.

Definition 10.2.6: Network Injection

Iniezione di messaggi di protocollo fasulli (routing, gestione) verso AP esposti, con l'obiettivo di degradare le prestazioni o reindirizzare il traffico.

10.2.1 Attacchi passivi vs attivi

Definition 10.2.7: Attacco passivo

L'attaccante ascolta e basta, senza alterare il flusso. Esempi: *eavesdropping* (intercettazione) e *traffic analysis* (analisi dei metadati anche senza decifrare). Difficili da rilevare perché non lasciano tracce.

Definition 10.2.8: Attacco attivo

L'attaccante modifica, inietta o blocca il traffico. Rientrano in questa categoria: *masquerade*, *data modification*, *denial of service*, *replay attack*.

10.3 Attacchi al livello radio

10.3.1 Jamming

Il **jamming** è la forma più elementare di DoS wireless: trasmettere rumore o un segnale continuo sulla stessa frequenza della rete bersaglio.

Example 10.3.1 (OOK (On-Off Keying))

Nella modulazione OOK il bit 1 corrisponde alla portante accesa, il bit 0 alla portante spenta. Se l'attaccante tiene la portante **sempre accesa** (trasmette sempre "1111...") il ricevitore non distingue più i dati legittimi dal rumore. È come urlare di continuo mentre qualcuno cerca di sentire un sussurro.

Note:

Questo attacco richiede **molta energia**, proporzionale alla banda da disturbare.

10.3.2 Frequency Hopping Spread Spectrum (FHSS)

Per rendere il jamming molto più costoso si usa banda larga: invece di una singola frequenza portante, se ne usano **molte**, saltando da una all'altra secondo una sequenza pseudo-casuale (PRNG) concordata a priori tra Tx e Rx.

Proposition 10.3.1 Robustezza di FHSS

Un jammer che vuole disturbare la comunicazione deve disturbare **tutte** le frequenze contemporaneamente, richiedendo un'energia enormemente maggiore. In caso di collisione/jamming su una singola portante si perde solo la porzione di informazione trasmessa su quella frequenza.

Vantaggi:

- Migliora la sicurezza
- Riduce le interferenze
- Connessioni stabili
- Usato in Bluetooth, IoT, applicazioni militari

Note:

Conoscere il seed del PRNG di FHSS equivale a poter intercettare tutta la comunicazione: la sicurezza dipende interamente dalla segretezza del seed.

10.3.3 Eavesdropping / Sniffing

In un ambiente radio aperto, fare eavesdropping è semplice conoscendo tre cose:

- la **frequenza** (o la sequenza di frequenze nel caso FHSS)
- la **modulazione** usata
- il **protocollo**

Con queste info basta un ricevitore SDR (Software-Defined Radio) nel raggio del segnale. Nessun accesso fisico richiesto.

10.3.4 Replay Attack

C'è una categoria di attacchi che non richiede nemmeno di conoscere modulazione o protocollo: il **replay attack**.

Definition 10.3.1: Replay Attack

Catturare una porzione di spettro radio in un momento e ritrasmetterla identica in seguito. Non serve capire cosa c'è dentro: basta registrare e riprodurre.

Note:

Il canale radio NON è protetto da questo attacco dai meccanismi di confidenzialità o integrità! Se reinoltro un messaggio cifrato e integro, viene considerato valido.

Example 10.3.2 (Cancelli automatici)

I cancelli controllati via radiocomando usano spesso OOK a 433 MHz. Un attaccante con un RTL-SDR (ricevitore da pochi euro) cattura il treno di impulsi mentre apre il cancello, poi lo riproduce per aprirlo lui stesso, senza mai capire cosa significhi il segnale.

10.3.5 Rolling Code

Per difendersi dal replay naive, i sistemi più moderni usano il **Rolling Code**.

Definition 10.3.2: Rolling Code

Trasmettitore e ricevitore condividono un PRNG. Ogni pressione del telecomando invia il codice corrente, poi il trasmettitore avanza al codice successivo. Il ricevitore accetta solo se il codice è consistente con la sua posizione nel PRNG, e avanza anche lui.

Problema del disallineamento : se il trasmettitore invia ma il ricevitore non riceve (fuori portata), i due PRNG si disallineano. Soluzione: il ricevitore accetta codici in una **finestra** di N valori successivi a quello atteso (tipicamente ~ 100), poi sposta la finestra.

Question 1: Il Rolling Code risolve il replay?

No! Lo rende solo meno efficiente. Un attaccante può catturare vari codici prima che vengano usati dalla vittima, usarli in sequenza, poi ricatturarne altri. Per protezione robusta serve un **canale di ritorno**.

10.3.6 Challenge and Response

Il limite dei metodi precedenti è la **mancanza di feedback**: cancelli e telecomandi sono solo trasmettitori. Con **transceiver** (Tx + Rx) si può fare Challenge and Response.

Definition 10.3.3: Challenge-Response

1. Il client (autenticando) invia una richiesta di accesso.
2. Il server (autenticatore) genera un numero casuale fresco, il **nonce**, e lo invia al client.
3. Il client **cifra** il nonce con la chiave condivisa e risponde.
4. Il server decifra e verifica la corrispondenza.

Ogni sessione usa un nonce diverso, quindi un messaggio catturato non può essere riusato: il replay è neutralizzato strutturalmente.

10.3.7 Cifratura radio

Oltre che per autenticazione, la cifratura serve per **confidenzialità** e **integrità** dei dati in transito. Nel mondo radio si usano:

- **Cifrari a blocco**: es. AES (standard de facto oggi)
- **Cifrari a flusso**: es. **Kasumi** / **A5/3**, usato in 3G/4G GSM

Note:

Nel 2010 è stato pubblicato il "*sandwich attack*" (una variante di related-key attack) che ha rotto Kasumi permettendo di ricavare la chiave completa. Anche gli standard largamente adottati possono essere vulnerabili.

10.4 RFID

Definition 10.4.1: RFID (Radio Frequency Identification)

Sistema di identificazione basato su campi elettromagnetici. Composto da:

- **Tag**: transponder radio miniaturizzato attaccato all'oggetto. Eccitato dal campo del lettore, trasmette ID e dati.
- **Lettore**: genera il campo e riceve la risposta.
- **Sistema informatico**: gestisce i dati ricevuti.

I tag possono essere **passivi** (alimentati dal campo del lettore, niente batteria) o **attivi** (con batteria, raggio maggiore). Usato ovunque: badge aziendali, carte contactless, antitaccheggio, passaporti biometrici.

Example 10.4.1 (Badge Unibo)

Usano la tecnologia **EM410X**: ogni badge ha un ID univoco, associato nel DB dell'università all'account.

Il varco consulta il DB per verificare l'autorizzazione.

10.4.1 Cloning dei tag RFID

I badge EM410X sono venduti con ID fisso e non modificabile. **Come aggirare la protezione?**

Note:

Basta comprare badge *scrivibili* compatibili con lo stesso standard, leggere l'ID del badge vittima con un lettore RFID, e scriverlo sul badge vuoto. Clone funzionante. Rasoio di Occam / percorso di minima resistenza: non si rompe la crittografia, ci si procura lo stesso strumento.

10.4.2 Spoofing RFID

Se non ci sono badge scrivibili ma il sistema è privo di crittografia, si usa un **transceiver RFID programmabile** (es. Proxmark) per emulare il segnale del tag legittimo:

1. L'attacker intercetta i segnali emessi dai tag RFID con un lettore.
2. L'attacker usa il dispositivo specializzato per **imitare quei segnali**.

La difesa fondamentale contro questi attacchi è l'uso di protocolli crittografici nell'RFID (challenge-response con chiavi vere). Tecnologie come **MIFARE DESFire** sono molto più resistenti di EM410X.

10.5 IEEE 802.11 — Wi-Fi

Lo standard **IEEE 802.11**, comunemente noto come **Wi-Fi**, è la tecnologia wireless più pervasiva nel mondo informatico. Crea **Wireless LAN (WLAN)** interoperabili con le reti Ethernet (IEEE 802.3).

Esistono diversi sotto-standard:

- **IEEE 802.11g**: reti a 2.4 GHz
- **IEEE 802.11ac/ax**: reti a 5 GHz

10.5.1 Layer fisico

A livello fisico **802.11 non prevede protezioni**. Le onde radio si propagano in tutte le direzioni attraverso i muri. L'unica "protezione" fisica possibile sono materiali schermanti (vernici tipo **YSHIELD**, gabbie di Faraday) che attenuano la propagazione fuori dall'ambiente controllato.

10.5.2 Accesso al canale: CSMA/CA

Le reti 802.11 usano un canale condiviso $\Rightarrow$ gestiscono le collisioni con **CSMA/CA**.

Definition 10.5.1: CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)

Prima di trasmettere un nodo "ascolta" il canale:

- Se occupato, aspetta.
- Se libero, aspetta comunque un tempo casuale (**backoff**) prima di trasmettere.

Questo riduce la probabilità che due nodi trasmettano nello stesso istante.

Problema del Terminale Nascosto

Non tutti i nodi si "vedono" a vicenda: l'AP di solito ha visibilità di tutta la rete, ma i singoli nodi no.

Example 10.5.1 (Hidden terminal)

Tre nodi: A , B (AP), C . Sia A che C sono nel raggio di B ma non nel raggio l'uno dell'altro.

$$[A] - (AP : B) - [C]$$

C non sa quando A sta trasmettendo $\Rightarrow$ può trasmettere credendo il canale libero $\Rightarrow$ collisione sull'AP. È una criticità strutturale delle reti wireless.

Inter-Frame Space (IFS)

Dopo una collisione, tutti i nodi aspettano un **Inter-Frame Space (IFS)** prima di ritrasmettere. Il tempo totale è:

$$\text{DIFS} + \text{backoff}$$

dove il backoff è estratto uniformemente in $[1, CW]$ con $CW = \text{Contention Window}$. Gli IFS hanno diverse priorità (SIFS, DIFS, EIFS) per diversi tipi di frame:

- Spazi attendibili solo dall'access point
- Spazi dedicati ai client
- ecc.

Appropriazione del canale (DoS su CSMA/CA)

Question 2: Cosa succede se qualcuno non rispetta l'IFS dopo una collisione?

Parla sempre e solo lui! Appropriazione totale della rete.

Question 3: E se due terminali non rispettano gli IFS?

Nessuno può più parlare! DoS totale.

10.6 Meccanismi di "sicurezza" deboli

10.6.1 SSID Hiding — Security by Obscurity

Definition 10.6.1: SSID Hiding

Disabilitare il broadcast del SSID del router: il nome della rete non appare nella lista delle reti disponibili. Per accedervi bisogna conoscerne il nome.

Il problema è fondamentale: quando un dispositivo che conosce già quella rete si riavvicina, manda **probe request in broadcast** chiedendo "C'è la rete X?" **con il nome in chiaro**. Chiunque abbia la scheda Wi-Fi in monitor mode può vedere questi probe e ricavare il nome.

Note:

Classico esempio di **security by obscurity**: una misura che dà una falsa sensazione di sicurezza senza fornire protezione reale.

10.6.2 MAC Address Whitelisting

Un altro "meccanismo" comune: l'AP accetta connessioni solo da MAC specifici.

Il problema è che l'indirizzo MAC viene trasmesso **in chiaro** in ogni frame 802.11. Un attaccante in ascolto vede i MAC autorizzati e con un semplice comando (**macchanger** su Linux) cambia il proprio MAC:

```
[root@snark ~]# macchanger -m 11:22:33:44:55:66 virbr0
Current MAC:      52:54:00:b7:9a:84 (unknown)
Permanent MAC:   00:00:00:00:00:00 (XEROX CORPORATION)
```

Ancora security by obscurity.

10.6.3 Management Frames in chiaro

L'autenticazione e la confidenzialità sono cruciali: in una rete **open** tutto il traffico viaggia in chiaro nell'etere, non serve nemmeno un MitM per sniffare.

Tuttavia, anche in reti cifrate, 802.11 prevede che alcuni **management frames** viaggino **in chiaro** e senza autenticazione/anti-replay. Tra questi: **deauthentication** e **disassociation**.

Le fasi dell'associazione 802.11 sono:

1. **Authentication phase** (Request/Response)
2. **Association phase** (Request/Response)
3. **Normal communication**

Deauthentication / Dissociation Attack

L'attaccante può inviare un messaggio di deauthentication falsificato (con MAC sorgente dell'AP legittimo, non verificato) a un client. Il client si disconnette immediatamente $\Rightarrow$ DoS istantaneo e silenzioso. Viene anche usato come primo passo di attacchi più complessi (es. forzare la riconnessione per catturare l'handshake WPA).

Note:

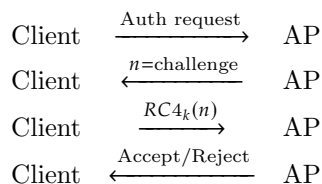
Soluzione: standard **IEEE 802.11w-2009**, noto come **Protected Management Frames (PMF)**. Obbligatorio in WPA3.

10.7 WEP — Wired Equivalent Privacy

Prima forma di autenticazione e confidenzialità in 802.11. Il nome era ambizioso: "privacy equivalente al cablato". La realtà è che **WEP è fondamentalmente rotto**, non va usato mai. Offre due modalità.

10.7.1 WEP — Shared Key

Tutti i dispositivi della rete condividono una chiave segreta. **Four-step challenge-response handshake:**



Scopo: il client dimostra di possedere la chiave.

Note:

Dannoso! Un attaccante che cattura lo scambio conosce sia il plaintext n (inviato in chiaro) sia il ciphertext $RC4(k, n)$. **Known Plaintext Attack (KPA)**: con abbastanza osservazioni ricava il keystream e quindi la chiave. Paradossalmente Shared Key è **meno sicura** dell'Open System.

10.7.2 WEP — Open System

Non c'è verifica con challenge. Chi si connette deve già conoscere il segreto comune, altrimenti non può decifrare i pacchetti dell'AP (e i suoi verranno scartati). L'autenticazione è implicita nella capacità di cifrare correttamente. Struttura del pacchetto:

$$| \text{IV} | \text{Key ID} | \underbrace{\text{Data} | \text{ICV}}_{\text{Encrypted}} |$$

WEP con RC4

Definition 10.7.1: RC4 in WEP

Stream cipher basato sulla chiave k e sul **Initialization Vector** v (a 24 bit), genera un keystream $RC4(v, k)$.

Per inviare il messaggio M da Alice a Bob:

1. Calcola checksum di integrità a 32 bit (CRC32): $c(M)$
2. Plaintext: $P = \{M, c(M)\}$
3. Cifra P : $C = P \oplus RC4(v, k)$
4. Trasmette $C' = v \parallel C$

Per decifrare il processo è invertito.

10.7.3 WEP — IV Collision

Theorem 10.7.1 Birthday Paradox sugli IV

Con soli 24 bit l'IV ha $2^{24} \approx 16$ milioni di valori possibili. Dopo $\sim 30\,000$ pacchetti la probabilità di collisione supera il 50%:

$$P_{\text{collision}} \approx 1 - e^{-\frac{30000^2}{2 \cdot 2^{24}}} \approx 1$$

Note:

Il birthday paradox dice che in un gruppo di 23 persone scelte a caso, almeno due condividono il compleanno con probabilità $\geq 1/2$.

Se due frame usano lo stesso IV, usano lo stesso keystream: lo XOR dei ciphertext è uguale allo XOR dei plaintext. Da lì si fanno **attacchi statistici** per ricavare contenuti e, con abbastanza dati, la chiave. Catturando pacchetti con IV noto si fa inoltre **replay** riciclando pacchetti vecchi.

Note:

Tool come **aircrack-ng** rompono WEP in pochi minuti su una rete trafficata.

10.8 IEEE 802.11i e WPA/WPA2

Per ovviare ai problemi di WEP è stato sviluppato **IEEE 802.11i**, implementato nella famiglia **Wi-Fi Protected Access (WPA)**, arrivata alla versione 3.

10.8.1 Modi di utilizzo

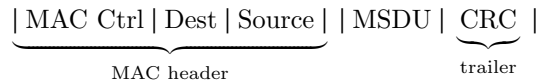
- **PSK (Pre-Shared Key, 256 bit)**: reti domestiche. La passphrase viene derivata in chiave a 256 bit tramite PBKDF2.
- **Enterprise**: autenticazione individuale tramite server RADIUS e protocollo EAP. Tipica di reti universitarie (ALMAWIFI, eduroam) e aziendali.
- **WPS**: sistema di autenticazione facilitato.

10.8.2 Schemi di protezione (802.11i)

IEEE 802.11i definisce due schemi per proteggere le **MPDU (MAC Protocol Data Unit)**:

- **TKIP (Temporal Key Integrity Protocol)**: compatibile con hardware WEP, oggi **deprecato**
- **CCMP (Counter Mode-CBC MAC Protocol)**: basato su **AES**, standard attuale

Formato MPDU:



10.8.3 TKIP

Progettato per richiedere solo **modifiche software** ai dispositivi WEP. Fornisce:

Definition 10.8.1: Integrità in TKIP — MIC

Aggiunge un **Message Integrity Code (MIC)** a 64 bit al frame MAC dopo il campo Dati. Il MIC è generato dall'algoritmo **Michael** a partire da:

- MAC di origine e destinazione
- campo Dati
- materiale della chiave

Definition 10.8.2: Confidenzialità in TKIP

Cifra MPDU + MIC usando **RC4**.

Temporal Key (TK) di 256 bit

- 2×64 bit $\Rightarrow$ usate con Michael per il MIC:
 - Una chiave per STA $\rightarrow$ AP
 - Un'altra per AP $\rightarrow$ STA
- Restanti 128 bit $\Rightarrow$ troncati per generare la chiave RC4 di cifratura

TKIP Sequence Counter (TSC)

Definition 10.8.3: TSC

Contatore di sequenza assegnato a ciascun frame. Due scopi:

- Incluso in ogni MPDU e protetto dal MIC $\Rightarrow$ protegge dai **replay attack**.
- Combinato con la TK per produrre una **chiave dinamica** che cambia a ogni MPDU $\Rightarrow$ rende molto più difficile la crittoanalisi.

10.8.4 CCMP — Counter Mode-CBC MAC Protocol

Destinato ai dispositivi 802.11 recenti con hardware dedicato. Fornisce:

- **Integrità**: MAC basato su CBC (Cipher Block Chaining) $\Rightarrow$ **CBC-MAC**
- **Confidenzialità**: AES in modalità **CTR (Counter)**

Note:

Dettagli:

- **Stessa chiave AES a 128 bit** sia per integrità che per confidenzialità.
- **Packet counter a 48 bit** come nonce $\Rightarrow$ previene replay (2^{48} valori, riutilizzo praticamente impossibile).

10.9 Attacchi a WPA e WPS

10.9.1 Attacchi a WPA-PSK

Anche WPA2 con CCMP non è immune se la passphrase è debole. Durante il **4-way handshake** di autenticazione vengono scambiati messaggi che permettono ad un attaccante in ascolto di catturare materiale crittografico sufficiente a tentare:

- **Attacchi bruteforce**
- **Attacchi a dizionario** (offline)

Si prova ogni passphrase candidata, si deriva la chiave con PBKDF2, si confronta con l'handshake catturato. Con GPU moderne e dizionari buoni, le password deboli cadono in minuti.

10.9.2 WPS (Wi-Fi Protected Setup)

WPS semplifica la connessione senza scambio di passphrase (una sorta di prova di accesso fisico). Metodologie:

- **Tasto fisico** (pressione di un pulsante)
- **Codice PIN a 8 cifre**

Attacchi a WPS

Tasto fisico: aggirabile con Rogue AP in posizione tattica.

PIN: problema molto più preoccupante.

- Il PIN è a 7 cifre effettive (l'ottava è un checksum) $\Rightarrow 10^7 = 10\,000\,000$ tentativi teorici.
- L'attaccante recupera il PIN in poche ore con brute force e da lì recupera la **PSK WPA/WPA2**.

Theorem 10.9.1 Errore di design di WPS

Il PIN viene controllato in **due blocchi**:

- prima le prime 4 cifre
- poi le altre 3 (solo se le prime 4 sono corrette)

Quindi i tentativi reali sono

$$10^4 + 10^3 = 11\,000 \approx 20 \text{ ore di brute force}$$

non 10^7 .

PixieDust

L'implementazione WPS su molti dispositivi embedded (TP-Link, D-Link, Netgear) usava un **generatore di nonce prevedibile**. L'attacco **PixieDust** riduce il brute force a **pochi minuti** catturando e analizzando la comunicazione.

Note:

Tool: `pixiewps` su Kali Linux (<https://www.kali.org/tools/pixiewps/>).

10.9.3 Rogue AP

Il settaggio del **nome della rete (SSID)** non è crittografato: solo l'accesso ad essa lo è. Questo permette attacchi **Rogue Access Point** con spoofing del nome: i client tentano l'accesso con la loro password e possono esporre handshake o credenziali all'attaccante.

Note:

Contromisura: WPA Enterprise con **certificati server** e autenticazione reciproca. Il client verifica l'identità dell'AP tramite certificato $\Rightarrow$ un AP fasullo non passa la verifica.

10.10 KRACK e WPA3

10.10.1 KRACK — Key Reinstallation AttaCK

Attacco del 2017, uno dei più eleganti e importanti degli ultimi anni. Sfrutta una vulnerabilità nel **4-way handshake di WPA2**, non nella crittografia in sé ma nel protocollo di gestione delle chiavi.

Definition 10.10.1: KRACK

Nel 4-way handshake viene negoziata una chiave di sessione e un nonce. Se il client non invia un ACK del terzo messaggio, il server ritrasmette. L'attaccante manipola questo meccanismo per far sì che il client **reinstalli la stessa chiave**, reimpostando il nonce al valore iniziale.

Proposition 10.10.1 Perché è grave

Se il nonce viene riusato con la stessa chiave, si ottiene lo **stesso keystream** per messaggi diversi. Come in WEP, questo rompe la confidenzialità. In modalità CTR (CCMP/AES) il riuso del nonce permette di decifrare e in alcuni casi forgiare pacchetti.

Note:

Da <https://www.krackattacks.com/>: "In a key reinstallation attack, the adversary tricks a victim into reinstalling an already-in-use key. [...] When the victim reinstalls the key, associated parameters such as the incremental transmit packet number (i.e. nonce) and receive packet number (i.e. replay counter) are reset to their initial value. Essentially, to guarantee security, a key should only be installed and used once."

KRACK è stato il catalizzatore dell'adozione di **WPA3**.

10.10.2 WPA3

WPA3 (2018) risolve le principali debolezze di WPA2:

Definition 10.10.2: SAE (Simultaneous Authentication of Equals)

Sostituisce il PSK nelle reti domestiche. Variante del **Dragonfly Key Exchange** (RFC 7664), basato su Diffie-Hellman. Offre **forward secrecy**: anche se un attaccante registra traffico cifrato oggi e in futuro scopre la passphrase, non può decifrare il traffico passato, perché le chiavi di sessione dipendono dai nonce freschi dell'handshake.

Vantaggi di WPA3:

- **SAE** $\Rightarrow$ attacchi a dizionario offline molto più difficili
- **Forward secrecy** $\Rightarrow$ traffico passato resta sicuro anche se la password viene scoperta dopo
- **PMF obbligatori** $\Rightarrow$ l'attacco di deauthentication non funziona più
- **Resistenza al KRACK**: il protocollo di handshake è stato riprogettato

Lato negativo: richiede hardware recente, non sempre retrocompatibile con dispositivi vecchi.

10.11 Riepilogo e concetti chiave

10.11.1 Evoluzione della sicurezza Wi-Fi

Standard	Cifratura	Integrità	Auth	Problemi
Open	—	—	—	Tutto in chiaro
WEP Shared	RC4	CRC32	Challenge	KPA, IV collision, replay
WEP Open	RC4	CRC32	PSK	IV collision (birthday)
WPA (TKIP)	RC4 + TSC	Michael (MIC)	PSK / Enterprise	RC4 obsoleto, deprecato
WPA2 (CCMP)	AES-CTR	CBC-MAC	PSK / Ent. / WPS	KRACK, PSK brute, WPS
WPA3 (SAE)	AES-GCMP	—	Dragonfly/SAE	Richiede HW recente

10.11.2 Concetti chiave

1. **Il canale radio è intrinsecamente insicuro:** confidenzialità e integrità non bastano, serve **anti-replay** (nonce, sequence counter).
2. **Security by obscurity non funziona:** SSID hiding e MAC whitelist sono aggirabili con sniffing.
3. **PRNG prevedibili distruggono la sicurezza:** es. PixieDust su WPS.
4. **Riuso di chiavi/nonce è fatale:** IV collision in WEP, KRACK in WPA2. Una chiave va installata e usata **una volta sola**.
5. **Jamming è sempre possibile a livello fisico:** FHSS mitiga ma non elimina.
6. **Anche i management frames vanno protetti:** IEEE 802.11w (PMF) risolve i deauth attack.
7. **Autenticazione reciproca:** il client deve verificare anche l'identità dell'AP (certificati) $\Rightarrow$ blocca i Rogue AP.
8. **Forward secrecy:** compromissione futura della password non rende decifrabili comunicazioni passate.
9. **Ogni evoluzione dello standard è stata guidata da attacchi reali:** WEP $\rightarrow$ WPA $\rightarrow$ WPA2 $\rightarrow$ WPA3.

10.11.3 Strumenti e riferimenti menzionati

- Wireshark — sniffing di traffico
- macchanger — spoofing MAC
- Proxmark — spoofing RFID
- pixiewps (Kali Linux) — attacco PixieDust a WPS
- aircrack-ng — rottura di WEP
- KRACK attacks — <https://www.krackattacks.com/>
- RFC 7664 — Dragonfly Key Exchange
- IEEE 802.11w-2009 — Protected Management Frames

Chapter 11

Exploits and Patches

Chapter 12

Attacchi di Buffer Overflow

12.1 Introduzione

12.1.1 Buffer overflow attacks

Il software è noto per avere **bug**:

- Funzionalità che non vanno come previsto o non funzionano affatto.
- **Crash** che causano inaffidabilità e perdita di dati.

Note:

Per un attacker, i **bug** nel software sono delle opportunità.

12.1.2 Exploits

Definition 12.1.1: Exploit

Bug del software trasformati in armi: l'utilizzo di errori di programmazione a vantaggio di un utente malintenzionato.

Usi tipici degli exploit:

- Aggirare i controlli di **autenticazione** e **autorizzazione**
- **Elevare i privilegi** (ad admin o root)
- Dirottare i programmi per eseguire **codice non intenzionale** / **arbitrario** / **shellcode**
- Consentire l'**accesso persistente e non autorizzato** ai sistemi

12.1.3 Storia degli incidenti più importanti

Anno	Evento
1988	Il Morris Internet Worm usò un exploit di buffer overflow in fingerd .
1995	Buffer overflow in NCSA httpd 1.3 pubblicato sulla mailing list Bugtraq da Thomas Lopatic.
1996	Aleph One pubblica <i>Smashing the Stack for Fun and Profit</i> su <i>Phrack</i> #49.
2001	Worm Code Red : buffer overflow in Microsoft IIS 5.0.
2003	Worm Slammer : buffer overflow in Microsoft SQL Server 2000.
2004	Worm Sasser : buffer overflow nel LSASS di Windows 2000/XP.

12.2 Definizione di buffer overflow

Definition 12.2.1: Buffer Overflow (NISTIR 7298, 2019)

Una condizione su un'interfaccia in cui è possibile inserire **più input in un buffer** (o area di conservazione dei dati) rispetto alla **capacità assegnata**, sovrascrivendo altre informazioni.

Gli avversari sfruttano tale condizione per mandare in crash un sistema o per inserire codice appositamente predisposto che consente loro di ottenere il controllo del sistema.

Note:

Detto anche **buffer overrun** o **buffer overwrite**.

Esempio — Buffer da 8 byte : buffer che dovrebbe contenere USERNAME, ma riceve USERNAME12:

Buffer (8 bytes)								Overflow	
U	S	E	R	N	A	M	E	1	2
0	1	2	3	4	5	6	7	8	9

I byte 1 e 2 finiscono **oltre il buffer**, sovrascrivendo memoria adiacente.

12.3 Esecuzione di un programma

12.3.1 I compilatori

[C] → [Front End] → [IR] → [Back End] → [ASM/BIN]

- I computer **non eseguono codice sorgente** (Java, C++, ...).
- Eseguono **codice macchina** (machine code).
- I **compilatori** traducono il codice sorgente in codice macchina.
- L'**assembly** è codice macchina leggibile dall'uomo.

12.3.2 Esempio C → x86-64

```
#include <stdio.h>
int main(int argc, char** argv) {
    int i;
    if (argc > 1) {
        for (i = 1; i < argc; ++i) {
            puts(argv[i]);
        }
    }
    else {
        puts("Hello world");
    }
    return 1;
}
```

In assembly x86-64 (estratto):

```
push    rbp
mov     rbp, rsp
sub     rsp, 0x20
mov     DWORD PTR [rbp-0x14], edi
mov     QWORD PTR [rbp-0x20], rsi
cmp     DWORD PTR [rbp-0x14], 0x1
```

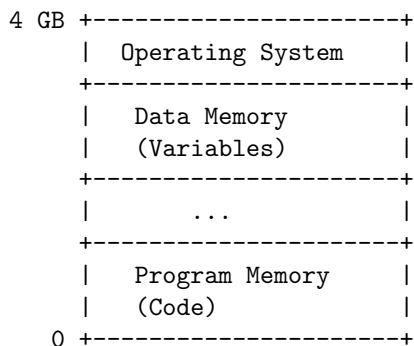
```

jle    400578 <main+0x4b>
...
call   400410 <puts@plt>
...
leave
ret

```

12.3.3 Memoria del computer

I **programmi in esecuzione** sono presenti nella memoria. Layout (Virtual Memory ~ 4 GB):



- **Program Memory:** il codice del programma.
- **Data Memory:** variabili, costanti e altre cose necessarie.
- **OS Memory:** sempre disponibile per le **system calls** (aprire un file, eseguire un altro programma, stampare, ecc.).

12.3.4 Memoria del programma e Instruction Pointer (IP)

Definition 12.3.1: Instruction Pointer (IP / PC)

Registro della CPU che tiene traccia dell'istruzione corrente. Detto anche *Program Counter*.

```

0: integer count(string s, character c) {
    integer counter=0;
    integer pos;
1:   for (pos = 0; pos < length(s); pos = pos + 1) {
2:     if (s[pos] == c) counter = counter + 1;
3:   }
4:   return counter;
5: }

6: void main(integer argc, strings argv) {
7:   count("testing", "t"); // should return 2
8: }

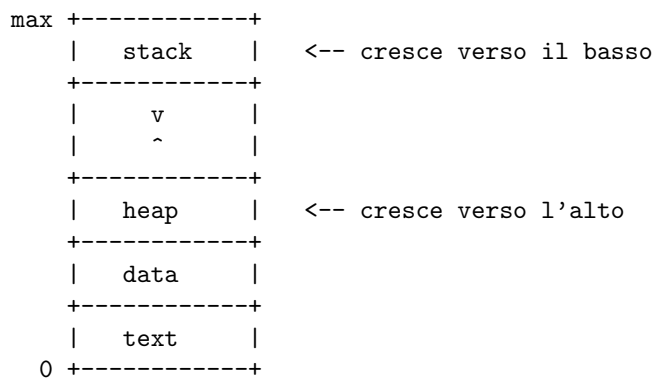
```

L'IP punta inizialmente alla riga 7 (chiamata di `count`). Il codice sta in Program Memory; le variabili (`counter`, `pos`) e le costanti ("`testing`", "`t`") sono salvate nella Data Memory.

12.4 Lo Stack e la chiamata di funzione

12.4.1 Lo Stack

La **memoria dati** è organizzata usando una struttura specifica: lo **stack**.



Definition 12.4.1: Stack frame

Ogni **funzione**, quando viene eseguita, ottiene un **frame** sullo stack.

- Frame **creato** quando viene chiamata la funzione, **distrutto** quando esce (exit/return).
- Contiene le **variabili locali**.
- Contiene le informazioni sul **flusso di controllo**: return address, old frame pointer.
- **Lo stack cresce verso il basso**.

12.4.2 Chiamata di una funzione

Sia P il chiamante (*calling*) e Q il chiamato (*called*) da P .

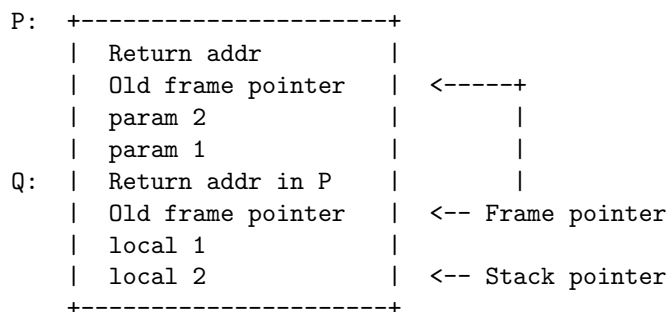
Lato chiamante (P)

1. Pusha i **parametri** per la funzione Q sullo stack (tipicamente in **ordine inverso** di dichiarazione: param 2, poi param 1).
2. Esegue l'istruzione `call` che invoca la funzione target $\Rightarrow$ pusha l'**indirizzo di ritorno** (Return addr in P) sullo stack.

Prologo della chiamata (Q)

3. Pusha il **valore corrente del frame pointer** (che punta al frame del chiamante).
4. Imposta il **frame pointer** al valore corrente dello stack pointer $\Rightarrow$ identifica la **nuova posizione del frame** per Q .
5. **Alloca spazio per le variabili locali** muovendo lo stack pointer verso il basso.
6. Esegue il **corpo** della funzione chiamata.

Layout del frame durante l'esecuzione di Q



Uscita di Q

7. Imposta lo **stack pointer** al valore del frame pointer (scartando lo spazio delle variabili locali).
8. Pop dell'**old frame pointer** (ripristina il link al frame del chiamante).
9. Esegue l'istruzione **ret**, che pop l'indirizzo salvato e restituisce il controllo a *P*: il **return address** è l'indirizzo dell'istruzione in *P* immediatamente dopo la **call**.

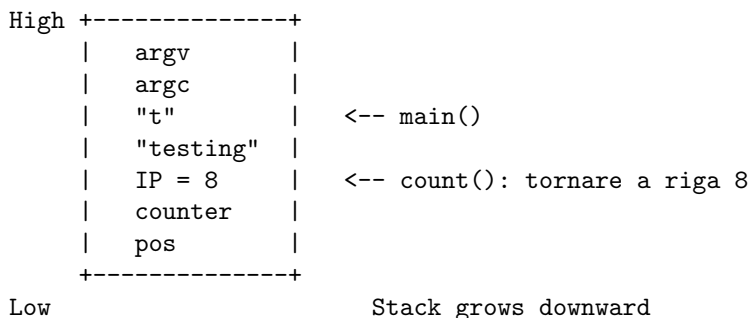
Ripresa in P

10. Pop dei **parametri** della funzione chiamata dallo stack.
11. Continua l'esecuzione con l'istruzione successiva alla chiamata.

12.4.3 Esempio di Stack Frame

```
0: string count(string s, character c) {
    integer counter;
    integer pos;
    ...
5: }
6: void main(integer argc, strings argv) {
7:   count("testing", "t");
8: }
```

Lo stack frame durante l'esecuzione di `count`:



Note:

Concetto chiave: il frame deve tenere traccia dell'**IP successivo** (return address): la CPU lo userà a fine funzione per tornare al chiamante.

12.4.4 Esempio di Ricorsione

```
0: integer r(integer n) {
1:   if (n > 0) r(n - 1);
2:   return n;
3: }
4: void main(integer argc, strings argv) {
5:   r(3); // should return 3
6: }
```

Stack risultante (dall'alto verso il basso):

```
main()  -> IP = ..., argv=3
r(3)    -> IP = 6,  n-1 = 2
r(2)    -> IP = 2,  n-1 = 1
r(1)    -> IP = 2,  n-1 = 0
r(0)    -> IP = 2,  0
```

Ogni chiamata ricorsiva impila un nuovo frame.

12.4.5 Review

1. I programmi in esecuzione sono nella **memoria (RAM)**.
2. Il codice assembly è nella **program memory**. La CPU tiene traccia della prossima istruzione nel registro **IP / PC**.
3. La **memoria dati** è strutturata come una **pila di frame**:
 - Ogni invocazione di funzione (**call**) aggiunge un frame.
 - Ogni frame contiene: **IP salvato** (e l'indirizzo del frame precedente) e le **variabili locali**.

12.5 Stack Buffer Overflow

Definition 12.5.1: Stack Buffer Overflow

Si verifica quando il **buffer** si trova nello **stack**, solitamente come variabile locale nello stack frame di una funzione. Detto anche **stack smashing**.

Più formalmente: un processo (programma in esecuzione) tenta di memorizzare dati **oltre i limiti** di un buffer (sullo stack) di **dimensione fissa**, sovrascrivendo le posizioni di memoria adiacenti.

Note:

Visti per la prima volta nel **Morris Internet Worm (1988)**, sfruttando la funzione C `gets()` nel daemon `fingerd`.

12.5.1 Un esempio semplice di programma vulnerabile

```
0: void print(string s) {
    // only holds 32 characters, max
    string buffer[32];
1:  strcpy(buffer, s);    // copy s into buffer
2:  puts(buffer);        // print buffer to stdout
3: }

4: void main(integer argc, strings argv) {
5:   for (; argc > 0; argc = argc - 1) {
6:     print(argv[argc]);
7:   }
8: }
```

Output normale:

```
$ ./print Hello World
World
Hello
```

```
$ ./print arg1 arg2 arg3
arg3
arg2
arg1
```

Dov'è il problema? `strcpy(buffer, s)` non controlla la lunghezza dell'input!

```
#include <cstring>
char *strcpy(char *to, const char *from);
```

Da man-page: *strcpy()* does not perform bounds checking, and thus risks overrunning from or to. For a similar (and safer) function that includes bounds checking, see `strncpy()`.

12.5.2 Cosa succede se s è più lunga di 32 caratteri?

Il buffer di 32 byte viene riempito, e i caratteri eccedenti **sovrascrivono**:

- l'old frame pointer
- il return address salvato (IP = 7)
- potenzialmente i parametri/dati di main()

```
High +-----+
      |  argv      |
      |  argc      |
      |  IP = ... (main) |
      |  IP = 7 (print) | <-- sovrascritto!
      |  Data from argv | <-- buffer riempito oltre i limiti
Low  +-----+
```

12.5.3 Crash del programma e DoS

Quando `print()` tenta di trasferire il controllo all'indirizzo di ritorno (IP = 7), salta a una **posizione di memoria illegale** ⇒ **Segmentation Fault** ⇒ terminazione anomala.

Note:

Fornire semplicemente un **input casuale** porta al crash del programma. Poiché il programma si è bloccato non può più fornire la funzione/servizio: nella sua forma più semplice, uno stack overflow è una forma di **Denial-of-Service (DoS)**.

12.6 Smashing the Stack

12.6.1 Idea chiave

- I bug di buffer overflow possono **sovrascrivere gli IP salvati** ⇒ crash del programma.
- **Idea chiave 1:** sostituire l'IP salvato ⇒ può puntare ovunque l'attaccante voglia. **Ma dove?**
- **Idea chiave 2:** riempire il buffer con **codice dannoso**.
 - Il codice macchina è solo una stringa di byte.
 - Cambia l'IP per puntare al **codice dannoso (shellcode)** sullo stack.

12.6.2 Exploit v1 — layout di memoria

Indirizzi di memoria di esempio:

```
:1000 +-----+
      |  argv      |
:996  +-----+
      |  argc      |
:992  +-----+
      |  IP = ... (main) |
:988  +-----+
      |  IP = 7 (print) | <-- da sovrascrivere
:984  +-----+
      |            |
      |  buffer (32 B) |
:952  +-----+
```

12.6.3 Exploit v1 — sovrascrittura

L'attaccante inietta:

- nel buffer (a partire da :952): **shellcode** (codice malicious).
- dove si trova IP = 7 (a :984): IP = 952 $\Rightarrow$ punta all'inizio dello shellcode.

```
:1000 +-----+
      |   argv   |
:996  |   argc   |
:992  |   IP = ... |
:988  |   IP = 952 | <-- sovrascritto!
:984  +-----+
      |          |
      | Malicious code / |
      | Shellcode       |
:952  +-----+
```

Quando `print()` ritorna, salta a :952 ed esegue lo shellcode.

12.7 Shellcode

Definition 12.7.1: Shellcode

Componente essenziale di molti attacchi di buffer overflow: il **codice fornito dall'attaccante** a cui si trasferisce l'esecuzione, spesso salvato nel buffer in overflow.

La sua funzione classica è quella di trasferire il controllo a un **interprete della riga di comando dell'utente, o shell**, dando accesso a qualsiasi programma disponibile sul sistema **con i privilegi del programma attaccato**.

12.7.1 Caratteristiche

- Lo shellcode è semplicemente **machine code**: una serie di valori binari corrispondenti alle istruzioni macchina e ai dati che implementano la funzionalità desiderata.
- È **specifico per architettura del processore e sistema operativo**, perché deve interagire con le system call.

Esempio — Shellcode minimale (in C):

```
{
    // execute a shell with the privileges of the vulnerable program
    exec("/bin/sh", NULL);
}
```

12.7.2 Restrizioni generiche sul contenuto

Theorem 12.7.1 Vincoli sullo shellcode

Uno shellcode pratico deve soddisfare tre vincoli:

1. **Position-Independent**: deve poter essere eseguito indipendentemente da dove si trovi in memoria. Possono essere usati solo riferimenti **relativi**, offset rispetto all'IP corrente.
2. **No NULL bytes**: in C la stringa termina con NULL ($\Rightarrow$ `strcpy` si fermerebbe!). L'unico NULL ammesso è alla **fine**, dopo tutto il codice e l'indirizzo di ritorno sovrascritto.
3. **Sopravvivenza alle trasformazioni**: cosa succede se il programma decifra/converte la stringa

prima di copiarla? Se trasforma minuscole in maiuscole? Lo shellcode deve essere costruito per evitare o tollerare queste modifiche.

12.7.3 Un classico shellcode per attacco execve

Codice C equivalente:

```
int main(int argc, char *argv[]) {
    char *sh;
    char *args[2];
    sh = "/bin/sh";
    args[0] = sh;
    args[1] = NULL;
    execve(sh, args, NULL);
}
```

Codice in assembly x86 (con commenti):

```
        nop
        nop
        jmp find          // jump to end of code
cont:   pop %esi           // pop address of sh off stack into %esi
        xor %eax, %eax    // zero contents of EAX
        mov %al, 0x7(%esi) // copy zero byte to end of string sh
        lea (%esi), %ebx   // load address of sh into %ebx
        mov %ebx, 0x8(%esi) // save address of sh in args[0]
        mov %eax, 0xc(%esi) // copy zero to args[1]
        mov $0xb, %al      // copy execve syscall number (11) to AL
        mov %esi, %ebx     // copy address of sh into %ebx
        lea 0x8(%esi), %ecx // copy address of args to %ecx
        lea 0xc(%esi), %edx // copy address of args[1] to %edx
        int $0x80          // software interrupt to execute syscall
find:   call cont          // call cont -- saves next address on stack
sh:     .string "/bin/sh"
args:   .long 0
        .long 0
```

12.7.4 Generare shellcode

Tool comuni:

- Metasploit Project — <https://www.metasploit.com/>
- pwntools — <https://docs.pwntools.com/en/stable/>

12.8 NOP Sled

12.8.1 Hitting the Target — il problema

Question 4: Come possiamo indovinare in modo affidabile l'indirizzo dello shellcode?

- L'indirizzo dello shellcode deve essere **indovinato esattamente**: l'IP deve saltare all'inizio preciso dello shellcode.
- **Gli indirizzi dello stack cambiano spesso**: cambiano ogni volta che un programma viene eseguito.

Trucco: rendere il bersaglio **ancora più grande** così è più facile da colpire.

12.8.2 NOP Sled

Definition 12.8.1: NOP Sled

La maggior parte delle CPU supporta istruzioni **no-op**: istruzioni semplici, da un byte, che non fanno nulla. Su Intel x86 l'opcode 0x90 è il NOP.

L'idea è di creare una *slitta* di NOP davanti allo shellcode: funge da grande **rampa**. Se l'IP atterra in un punto **qualsiasi** della rampa, esegue NOP finché non raggiunge lo shellcode.

12.8.3 Exploit v2 con NOP sled

Buffer più grande (128 byte) consente:

```
:1000 +-----+
      |   argv   |
:996  |   argc   |
:992  |   IP = ...|
:988  +-----+
:984  |   IP = 900| <-- punta nel mezzo del NOP sled
      +-----+
      | Malicious code |
      | (shellcode)    |
      +-----+
      |               |
      |   NOP sled     |
      | (0x90 0x90 ...) |
:856  +-----+
```

Anche se l'attaccante "indovina male" e atterra in un punto qualsiasi del NOP sled, l'esecuzione *scivola* fino allo shellcode.

12.9 Funzioni C non sicure e altre forme di overflow

12.9.1 Routine comuni della libreria C non sicure

Funzione	Descrizione
gets(char *str)	Read line from standard input into str
sprintf(char *str, char *fmt, ...)	Create str according to supplied format and variables
strcat(char *dest, char *src)	Append contents of src to dest
strcpy(char *dest, char *src)	Copy contents of src to dest
vsprintf(char *str, char *fmt, va_list ap)	Create str according to format and variables

Note:

Tutte queste **non controllano i limiti** del buffer di destinazione.

12.9.2 Altre forme di attacchi di overflow

- **Heap Overflows**: overflow in memoria allocata dinamicamente.
- **Global Data Area Overflows**: overflow in variabili globali.
- **Format String Overflows**: sfruttamento di funzioni printf-like con format string controllata dall'utente.
- **Integer Overflows**: overflow numerico che produce dimensioni errate.

12.10 Difese contro i Buffer Overflow

12.10.1 Stack Canaries — StackGuard

Definition 12.10.1: Stack Canary

Il compilatore aggiunge un **valore speciale** (sentinel) sullo stack **prima di ogni IP salvato**.

- Il canary è impostato a un **valore casuale** in ogni frame.
- All'**uscita della funzione** il canary viene **controllato**.
- Se è stato alterato, il programma esce con un **errore**.

Esempio:

```
0: void print(string s) {
1:   __set_stack_canary(random());
2:   string buffer[32];
3:   strcpy(buffer, s);
4:   puts(buffer);
5:   __check_stack_canary()
6: }
```

Layout di memoria con canary:

```
:1000 +-----+
      |  argv          |
:996  |  argc          |
:992  |  IP = ...      |
:988  +-----+
      |  IP = 7        |
:980  +-----+
      |  canary = 189476 | <-- sentinel
      +-----+
      |                |
      |  buffer        |
:948  +-----+
```

Proposition 12.10.1 Effetto del canary contro lo smashing

Il malicious code sovrascrive il canary insieme all'IP. Al `__check_stack_canary()` la funzione rileva la modifica $\Rightarrow$ chiama `exit()` invece di tornare $\Rightarrow$ **l'attacco fallisce**.

12.10.2 Stack non eseguibile

Definition 12.10.2: NX / DEP (Data Execution Prevention)

Le CPU moderne impostano la memoria dello stack come **lettura/scrittura, ma non eXecute**.

Note:

Non evita i buffer overflow ma **impedisce** che lo shellcode venga inserito ed eseguito nello stack.

12.10.3 Address-Space Layout Randomization (ASLR)

Definition 12.10.3: ASLR

Difesa al **livello del sistema operativo**: **randomizzazione del layout della memoria**. Ogni volta che un programma viene caricato, la posizione del codice (program memory) e dei dati (data memory) cambia.

- Rende **più difficile** per l'attaccante indovinare la destinazione del buffer sullo stack.
- **Non impedisce lo sfruttamento, ma lo rende più difficile.**
- Supportato da tutti i moderni sistemi operativi.
- Funziona meglio quando la dimensione della memoria è grande (più entropia $\Rightarrow$ più difficile indovinare).

12.10.4 Write Safe Code!

Note:

Il messaggio importante è che **se i programmi non sono codificati correttamente** per proteggere le loro strutture dati fin dall'inizio, sono soggetti a potenziali attacchi.

Sebbene canary, NX e ASLR possano bloccare molti attacchi, alcuni — come la corruzione del valore di una variabile adiacente per alterare il comportamento del programma — **non possono essere bloccati se non con codice sicuro.**

Linee guida:

- Usare funzioni con bounds checking: `strncpy`, `snprintf`, `fgets` invece di `strcpy`, `sprintf`, `gets`.
- Validare gli input.
- Evitare linguaggi/funzioni che lasciano la responsabilità della memoria al programmatore quando possibile (es. usare Rust).